JavaScript, apprendre les bases du langage

Déclaration et portée des variables

Javascript permet de stocker des données à l'intérieur de structures appelées variables. Ces dernières peuvent contenir tout type de valeur et ne sont pas limitées dans leur changement, par exemple une variable contenant une chaîne de caractères pourra stocker plus tard une valeur numérique (bien que ce ne soit pas une bonne pratique).

Les variables ont une portée, çàd qu'elles sont accessibles dans l'espace dans lequel elles ont été définies ainsi que dans tout les autres sous-espaces (ou espaces imbriqués de types closures etc ...).

La syntaxe pour définir une variable est: '**var ma_variable**'. Un nom de variable valide comporte tout les caractères alphanumériques hormis les caractères spéciaux et doit impérativement commencer par une lettre.

```
// définit une variable contenant la chaîne hello world
var msg = "hello world";

function hello() {
    // ici la variable est accessible car elle a été définie
    // dans l'espace global qui est un espace parent de la fonction
    alert(msg);
}
```

Les tableaux

Javascript nous mets à disposition des structures appelées **Arrays**, ou tableau. Ces tableaux permettent de stocker une suite de valeurs au lieu d'une seule. Il est possible de stocker des valeurs de type complètement différents les unes à la suite des autres. Pour déclarer un tableau, il existe deux types de syntaxes, la syntaxe objet et la syntaxe JSON.

Pour accéder à un élement de tableau, il suffit d'utiliser la syntaxe suivante: '** tab[index]**', ou index représente la position d'un élément à l'intérieur du tableau. Les tableaux sont** '0-based**', cela signifie que le 1er élement se trouve en position 0, le 2ème en position 1 etc...

```
// déclare un tableau vide ...
var tab1 = [];  // syntaxe JSON
var tab2 = new Array(); // syntaxe objet

// déclare un tableau pré-rempli ...
var tab3 = [0, 80, "hello world"];

// affiche "hello world"
alert(tab3[2]);

// pousse un élement en 5ème position
alert(tab3[4]);
```

Opérateurs logiques et arithmétiques

Javascript introduit un certain nombre d'opérateur, un opérateur binaire nécéssite deux opérandes, un opérateur unaire n'en nécéssite qu'un seul, et un opérateur ternaire nécéssite trois opérandes.

- Les opérateurs d'affectation
- Les opérateurs de comparaison
- Les opérateurs arithmétiques
- Les opérateurs binaires
- Les opérateurs logiques
- Les opérateurs de chaînes
- Les opérateurs ternaire (conditionnel)
- L'opérateur virgule
- L'opérateur unaire
- L'opérateur relationnels

```
"Les opérateurs d'affectation",
("Les opérateurs de comparaison",
"Les opérateurs arithmétiques",
"Les opérateurs binaires",
"Les opérateurs logiques",
"Les opérateurs de chaînes",
"Les opérateurs ternaire (conditionnel)",
"L'opérateur virgule",
"L'opérateur unaire",
"L'opérateur relationnels")

//opérateurs arithmétiques:
alert(1 + 2); //addition
alert(1 * 10); // multiplication
alert(1 / 0.1); // division
alert(10 - 5); // soustraction
alert( 10 % 3 ); // modulo

// opérateurs logiques:

alert(10 == "10"); // égalité non stricte
alert( 10 === "10"); // égalité stricte
alert( 10 != "10" ); // non égalité non stricte
alert( 10 !== "10" ); // non égalité stricte
alert( (10==10) && (5==5)); // A est vrai et B est vrai
alert( (10==10) || (5==5)); // A est vrai ou B est vrai


//ternaire;
alert( (10==8) ? "yes" : "no");
```

Les boucles

Les boucles permettent de répéter des actions simplement et rapidement, il y en a différents types mais elles se ressemblent toutes au sens où elles répètent une action un certain nombre de foi

```
var i = 0, prop = null, obj = {"name":"Dark Vador", "job":"Sith Lord" };
// boucle for
for (i = 0; i < 10; i++) {
    console.log(i);
}

// boucle while
i = 0;
while (i < 10) {
    console.log(i++);
}

// boucle do while
i = 0;
do {
    console.log(i++);
}
while (i < 10)

//boucle for in
for (prop in obj) {
    console.log(prop, obj[prop]);
}
```

Déclaration et appel de fonctions

Une fonction est une suite d'instructions que le développeur pourra utiliser (on dit invoquer) quand il en aura besoin. Elle peut avoir un nom, recevoir des paramètres, et retourner une valeur. Javascript apporte son propre lot de fonctions mais il est possible de coder les siennes !

```
// cette fonction nommée saySomething affiche une boîte de dialogue
// contenant le message contenu dans le paramètre p_msg, elle fait appel
// à une fonction Javascript nommée alert.

function saySomething(p_msg) {
    alert(p_msg);
}

// cette fonction ne porte pas de nom, on dit qu'elle est anonyme.
// ici, elle est stockée dans une variable nommée myfunc
var myfunc = function () {
    alert("anonymous function");
};

/// invoque la fonction saySomething et affiche une boite de dialogue
saySomething("Hello World");

// invoque la fonction anonyme à l'aide de la variable myfunc
myfunc();
```

Gestion des erreurs et des exceptions

En Javascript, il est fréquent que des erreurs puissent survenir au sein du code, parfois elles n'ont (presque) aucune conséquence et le programme continue de fonctionner tant bien que mal. D'autres fois en revanche, il est tout à fait possible que le code plante complètement, soit à cause d'une étourderie du développeur (c'est plus fréquent qu'on ne le pense), soit parce qu'une erreur a été jeté volontairement.

Il est en effet possible d'interrompre volontairement le code en jetant un objet de type Error suite à un comportement prévu mais non voulu, pour cela il faut utiliser l'opérateur **throw**. Puisqu'il est possible de jeter des erreurs, il est possible de les attraper à l'aide du duo d'opérateurs **try ... catch**

```
// on essaie d'éxécuter le code contenu dans le bloc try ...
try {
    // ... code potentiellement à risque.
}
catch (error) {
    //  mais une erreur peut se produire et être jetée dans ce cas on l'attrape avec catch
    console.log(error);
}


// on peut également jeter nos propres erreurs à l'aide de l'opérateur throw
throw (new Error("This is an error, don't do that again !"));
```

LE JAVASCRIPT : UN LANGAGE ORIENTÉ OBJET

Développement Objet en JavaScript

Javascript n'est pas un langage orienté objet classique, en effet son paradigme objet (sa façon de le penser et l'implémenter si vous préférez) n'est pas celui d'un langage orienté objet par classe.

En fait Javascript est un **langage orienté objet par prototypage**, cela veut dire qu'il met l'accent sur la **décoration** d'objet, le **prototypage**, la **mutabilité** des objets. Dans un langage plus 'classique' comme Java, un objet aura un jeu de propriétés et de méthodes qui lui sont propres car définies sur une **classe**.

La notion de classe est absente en Javascript ES5 (en ES6 il s'agit avant tout de 'sucre syntaxique'), à la place, le développeur va devoir **créer des objets et leur ajouter/supprimer/modifier des propriétés 'à la volée'**, c'est ça un langage orienté objet par prototypage !

Comme il n'y a pas de classe à proprement parler, **il n'y a pas** non plus de mécanisme propres aux langages orientés objet par classe, comme **l'héritage**, **l'interfacage**, le cloisonnage par **namespaces** etc... Si le développeur souhaite profiter de ce genre de mécanismes en Javascript, il devra les **émuler à l'aide d'une bibliothèque** et/ou de fonctionnalités qu'il aura lui même crée.

En Javascript, même les fonctions sont des objets, ce qui tombe bien car du coup elles possèdent, comme tout les objets, d'une propriété nommée 'prototype' qui nous sera bien utile plus tard. En ES5, **pour émuler des classes, il est donc très commun d'utiliser des fonctions !**

Notion de constructeur

Pour **émuler le comportement des classes nous utilisons des fonctions en ES5**, et il se trouve que le corps (la suite des instructions contenues dans la fonction) de ce mêmes fonctions peut très bien jouer le rôle des fonction constructrice (ou constructeur).

Ainsi, une **fonction** va pouvoir nous servir à la fois de **pseudo classe ET de constructeur** pour de nouveaux objets crée grâce à elle, ce qui est plutôt pratique ! L'objet nouvellement crée disposera d'une propriété **constructor** qui fera ainsi référence à la fonction qui a servi à le créer, les objets 'classiques', crées à la volée, ont comme constructeur la classe Object.

```
function Personnage(param_name)
{
    // on peut traiter le paramètre comme si on était dans une
    // fonction constructrice d'un 'vrai' objet de langage orienté objet
    // par classe
}

// on crée un nouvel objet Personnage
var darkvador = new Personnage("dark vador");
```

Notion de destructeur

En Javascript, contrairement au constructeur, il n'existe pas de mécanisme pouvant s'apparenter à une fonction destructrice. En effet, les objets sont supprimés à la main par le développeur, ainsi que tout les objets qui lui appartiennent en passant leur valeur à null.

En faisant cela, Javascript va indiquer au Garbage Collector (ramasse miettes) que l'objet est éligible à la destruction, il va donc pouvoir libérer l'espace occupé en mémoire. Le développeur devra donc, s'il souhaite en avoir une, coder sa propre fonction destructrice selon ses propres conventions de nommage ET l'appeler explicitement avant de passer la valeur de l'objet à null.

```
function Personnage() {
    // constructeur
}

// exemple fonction destructrice
Personnage.prototype.destroy = function () {
    // libère les propriétés de l'objet
}

// on crée un objet de type Personnage
var anakin = new Personnage();

// invocation de destructeur + instance = null
anakin.destroy();
anakin = null;
```

Le mot clé prototype

En Javascript, toutes les données non scalaires (Array, List, Object, Function, String etc...) héritent en réalité de la classe Object et ce, même s'il n'y a pas de mécanisme d'héritage au langage à disposition du développeur.

Cela veut dire que même **une fonction est un objet**, un peu spécial certes, mais un objet quand même. En tant que tel, il **dispose donc des méthodes et propriétés** définies sur Object, notamment la **propriété prototype**.

Cette propriété contient un objet un peu spécial, car il va être utilisé par le moteur Javascript pour créer nos nouvelles instances à l'aide de nos pseudo classes. En effet, au moment de la création d'une instance, **le prototype de la fonction** qui a servi en tant que pseudo classe va être analysé par le **moteur Javascript**, et ce dernier va **copier l'ensemble des propriétés et méthodes** décrites sur la nouvelle 'instance'.

```
function Personnage(){}

// création d'une propriété name sur le prototype
Personnage.prototype.name = "no_name";

// crée un objet de 'type' Personnage
var santa = new Personnage();

// retourne no_name (valeur par défaut), la propriété existe !
console.log(santa.name);
```

Le mot clé this

Le mot clé this en Javascript fait référence au contexte d'éxécution en cours, ou à celui qui a invoqué la fonction/méthode dans laquelle on se trouve (et qui donc devient le contexte en cours). Cela peut créer quelques problèmes parfois, selon la façon dont une méthode a été invoquée, 'this' ne pointera pas nécessairement vers l'objet 'propriétaire' de la méthode.

```
function Personnage(p_name) {
    this.name = p_name; // this = objet nouvellement crée
}
Personnage.prototype.name = "no_name";

Personnage.prototype.sayHello = function () {
    console.log(this.name);
};

var perso = new Personnage("Black Beard");

// ici, grâce à la syntaxe 'à point', Javascript va déterminer que le
// contexte de la fonction est l'objet perso, this, pointera donc
// vers le bon objet au sein de la méthode sayHello
perso.sayHello();

// ici, on stocke une 'référence' à la méthode sayHello et on l'utilise
// en faisant cela, on a crée une variable myfunc au sein de l'espace global
// ce qui en fait automatiquement une de ses propriétés
var myfunc = perso.sayHello;

// ici, appeler myfunc directement ou window.myfunc revient au même,
// this pointera donc vers window au sein de la méthode sayHello
myfunc(); window.myfunc();
```

Héritage en Javascript

L'héritage est complètement absent en Javascript dans sa version ES5 (en ES6, il s'agit avant tout de sucre syntaxique). Cependant, il est tout à fait possible d'émuler l'héritage, il suffit pour cela de copier le prototype de la 'classe mère' sur le prototype de la 'classe fille' avant de définir les propriétés et méthodes propres à cette dernière.

Pour appeler des méthodes parentes, il suffit d'invoquer la méthode définie sur le prototype de la classe mère en précisant le contexte d'éxécution comme étant celui de la 'classe fille' (à l'aide de la méthode 'apply' ou 'call').

```
function Car() { }

Car.prototype.start = function () {
    console.log("Engine started");
}

function Deloreane() { }

// copie le prototype de Car sur le prototype de Deloreane
Deloreane.prototype = Object.create(Car.prototype).prototype;

// équivalent d'une surcharge
Deloreane.prototype.start = function () {
    // équivalent de super
    Car.prototype.start.apply(this);
    console.log("... 2.21 GigoWatts !");
}
```

Interfaces en Javascript

Il n'existe pas à proprement parler d'interface en Javascript, en fait il est nécessaire encore une fois d'émuler le comportement des vraies interfaces. L'une des façons simples de faire est de créer un objet (et non pas une fonction car on pourrait l'instancier) qui contient des méthodes vides et des propriétés de valeurs nulles.

Par la suite, il faudra copier l'ensemble des propriétés et méthodes de cette interface sur le prototype de la 'classe' qui souhaite l'implémenter, ainsi elle respectera de facto le contrat passé avec l'interface.

```
var IPersonnage = {
    sayHello: function(){},
    name: null
};

function Personnage(){}

// emulation d'une implementation d'interface IPersonnage
for( var prop in IPersonnage ){
    Personnage.prototype[prop] = IPersonnage[prop];
}
```

Classes abstraites en Javascript

Il n'existe pas de classes abstraites en Javascript, car il n'existe pas de classes tout court. L'une des façons d'émuler le comportement des classes abstraites est de créer une pseudo classe (fonction) qui va servir de classe abstraite et qui va vérifier si le constructeur est celui de la classe abstraite. Si oui, on jette une erreur, sinon tout est ok.

```
//
// création d'une pseudo classe abstraite
function AbstractPersonnage() {
    if (this.constructor == AbstractPersonnage)
        throw (new Error("don't instanciate abstract class !"));
}

AbstractPersonnage.prototype.name = "no_name";

function Personnage() {
    AbstractPersonnage.apply(this); // call super constructor
}

// héritage ...
for (var prop in AbstractPersonnage.prototype) {
    Personnage.prototype[prop] = AbstractPersonnage[prop];
}

new Personnage(); // tout est ok
new AbstractPersonnage(); // jette une erreur
//
```

L'objet window

L'objet window représente la fenêtre elle-même. La propriété document d'un objet window pointe vers le document DOM chargé dans cette fenêtre. L'objet window représentant la fenêtre correspondant à un document donné peut être obtenue à l'aide de la propriété document.defaultView.

L'objet window n'est pas partagé entre les onglets faisant partie de la même fenêtre. Certaines méthodes, comme window.resizeTo et window.resizeBy s'appliquent à la fenêtre entière et non à l'onglet spécifique auquel l'objet window appartient. En général, ce qui ne peut pas raisonnablement être limité à un onglet se rapporte à la fenêtre à la place.

Lorsqu'on crée une variable dans l'espace global en Javascript, on crée en vérité une nouvelle propriété pour l'objet window.

```
// créer une variable globale revient à ajouter une propriété à l'objet window
var myprop = "myvalue";
console.log(window.myprop);
```

L'objet document

Toute page web chargée dans un navigateur web a son propre objet document. Cet objet sert de point d'entrée au contenu de la page et apporte des fonctions générales au document. L'interface de l'objet document permet d'accéder à de nombreuses choses comme le type de document, son URL, l'arbre DOM (incluant des éléments tels que <body> ou encore <table>), tout en fournissant des méthodes pour créer et manipuler chacun des sous-nœuds du document.

Notez que certaines APIs du DOM ne sont pas disponibles dans tous les navigateurs web, pour différentes raisons :

- Obsolète : en cours de suppression dans les navigateurs web supportant cette méthode/propriété
- Non-standard: il s'agit soit d'une fonctionnalité expérimentale pas (encore?) acceptée par tous les fabricants, ou une fonctionnalité spécifique au code du navigateur (par exemple Mozilla a quelques APIs DOM créés pour la création d'add-ons et le développement d'applications).
- Ou il s'agit d'un standard emergeant ou terminé mais qui n'a pas (encore?) été implémenté par les navigateurs web.

De manière générale, tout les navigateurs modernes implémentent désormais une API DOM unifiée et supportée dans son intégralité, seuls les anciens navigateurs posent encore problème, c'est pour ça que des librairies telles que jQuery, spécialisées dans la manipulation transparente du DOM, sont encore majoritairement utilisées.

L'objet navigator

L'objet Navigator représente l'état et l'identité de l'agent utilisateur et peut être récupéré via la propriété en lecture seul window.navigator. De manière générale, l'objet navigator peut permettre au développeur de récupérer des informations telles que:

- Le navigateur utilisé (via la propriété userAgent)
- L'état de la batterie (via Battery API)
- La langue préférée de l'utilisateur
- ... Etc plein d'autres !

```
// affiche une chaîne de caractères donnant des informations
// sur le navigateur utilisé
console.log(window.navigator.userAgent);

// affiche la langue préférée de l'utilisateur
console.log(window.navigator.language);

// retourne un objet de type BatteryManager
console.log(window.navigator.battery);
```

L'objet screen

En Javascript, l'objet Screen, accessible via la propriété screen de l'objet window (window.screen) nous sert à récupérer des informations à propos de l'écran sur lequel s'affiche la fenêtre courante.

```
// affiche les hauteur/largeur de l'écran en pixels
console.log( window.screen.height );
console.log( window.screen.width );

// affiche l'orientation de l'écran
console.log( window.screen.orientation );

// bloque/débloque l'orientation de l'écran
window.screen.lockOrientation();
window.screen.unlockOrientation();
```

L'objet location

L'objet Location (accessible via window.location) contient des informations concernant l'URL actuelle du document et fournit des méthodes pour modifier cette URL. Cette propriété peut être utilisée pour charger une autre page.

```
// affiche l'hôte, çàd une châine de la forme: domaine:port
console.log(window.location.host);

//affiche les paramètres çàd une chaîne: ?param=value&param=value
console.log(window.location.search);

//affiche l'url entière
console.log(window.location.href);

//recharge la page courante
window.location.reload();

//redirige vers la page monsite.fr
window.location.assign("http://monsite.fr");

//redirige vers la page monsite.fr, n'apparaîtra pas dans l'historique
window.location.replace("http://monsite.fr");
```

L'objet History

La propriété en lecture seule window.history renvoie une référence à l'objet History, qui offre la possibilité de manipuler l'historique de session du navigateur (pages visitées dans l'onglet ou le cadre dans lesquels la page courante est chargée).

Pour des raisons de sécurité, l'objet History n'autorise pas l'accès au URL d'autres pages présentes dans l'historique de la session mais autorise d'y naviguer.

Il n'y a pas de possibilité de nettoyer l'historique de la session ou de désactiver les boutons précédent et suivant. La solution éventuelle la plus proche serait la méthode location.replace(), permettant de remplacer l'élément actuel de l'historique de la session par l'URL fournie.

```
// retourne en arrière d'un cran dans l'historique
window.history.back();

// avance d'un cran dans l'historique
window.history.forward();

// retourne en arrière de 2 crans dans l'historique
window.history.go(-2);

// retourne la longueur de l'historique
console.log( window.history.length );
```

Création d'objet au format JSON

JSON (JavaScript Object Notation) est un format d'échange de données. Bien qu'il n'en soit pas un sous-ensemble au sens strict, JSON ressemble fortement à un sous-ensemble de la syntaxe JavaScript. Le format JSON est accepté par de nombreux langages de programmation.

JSON permet de représenter des nombres, des booléens, des chaînes de caractères, la valeur null, des tableaux (séquences ordonnées de valeurs) et des objets (correspondances chaînes-valeurs) composés de ces valeurs (ou d'autres tableaux ou objets). Il ne permet pas, nativement, de représenter des données plus complexes comme des fonctions, des expressions rationnelles ou des dates.

```
//objet au format json sous forme de paires clés/valeurs
{
   "name":"dark vador",
   "age": 50,
   "isJedi":false
}

//tableau au format json
["dark vador", 80, { "mykey":"myvalue"}, true, null ]
```

ECMASCRIPT 2015 ET AU DELÀ

Bonnes pratiques Ecmascript 5

Javascript est un langage qui évolue en permanence, ceci dit, dans sa version Ecmascript 5, les bonnes pratiques restent globalement inchangées. Les voici en versions condensées, et bien entendu cette liste n'est en rien exhaustive.

- Eviter les variables globales
- Déclarer les variables locales à l'aide du mot clé "var"
- Le mode strict doit être utilisé
- Eviter la fonction eval()
- Organiser son code en module
- Une expression doit toujours se terminer par un point-virgule
- Le code doit être découpé en plusieurs fichiers

... Et bien d'autres que vous pourrez retrouver par exemple à l'adresse suivante : https://maxlab.fr/javascript/bonnes-pratiques-javascript-pour-lentreprise

ES7/ES6/ES2015, présentation générale.

ECMAScript est un ensemble de normes concernant les langages de programmation de type script et standardisées par Ecma International dans le cadre de la spécification ECMA-262. Il s'agit donc d'un standard, dont les spécifications sont mises en œuvre dans différents langages de script, comme JavaScript ou ActionScript, ainsi qu'en C++ (norme 2011). C'est un langage de programmation orienté prototype.

Pour le langage Javascript, la norme ECMASCRIPT 5 est aujourd'hui la plus répandue, toutefois certains navigateurs, comme Chrome (Google Inc.), mettent déjà à disposition des développeurs une implémentation de Javascript respectant la norme ECMA 6 (ES2015).

Nommée ES2015, la dernière version d'ECMAScript a été publiée en juin 20156. Son support par les navigateurs évolue progressivement, mais il est possible d'utiliser un transcompilateur (tel que Babel.js8) vers ES5 pour développer dès aujourd'hui en ES6.

Première version publiée après le changement du processus de normalisation, l'ES7, ou ES2016, apporte peu de changements au langage. Le nouveau processus prévoit en effet de publier chaque année une nouvelle norme avec les ajouts qui ont eu lieu dans l'intervalle, afin d'éviter de publier des changements énormes comme ce fut le cas en 20159.

ECMASCRIPT 6 et 7, les nouveautés

Les normes ECMASCRIPT 6 et 7 arrivent avec leurs lots de nouvelles fonctionnalités notamment les suivants (mais pas que):

- Les mot-clés let et const
- Les templates et chaînes de caractères
- Les paramètres par défaut
- Les promesses
- Les fonctions lambda (arrow functions)
- Les classes et l'héritage

Nous allons détailler un peu plus chacune de ces fonctionnalités.

Les mot-clés let et const

Le mot clé **let** permet de déclarer une variable limitée à la portée d'un bloc, c'est-à-dire qu'elle ne peut être utilisée que dans le bloc où elle a été déclarée, ce qui n'est pas le cas avec var.

Le mot-clé **const** sert à définir une valeur immuable, accessible uniquement en lecture seule.

```
function whoIsTheSithLord() {
    // la valeur ne peut plus être modifiée
    const MOVIE = "STAR WARS";

    if (true) {
        // existe en dehors du bloc conditionnel
        var sith_lord = "Palpatine";
        //existe uniquement dans le bloc conditionnel
        let padawan = "Dark Vador";
    }

    console.log(sith_lord); // displays Palpatine
    console.log(padawan); // Uncaught Reference Error
    conole.log(MOVIE);
}

whoIsTheSithLord();
```

Templates Strings

Les "templates strings" permettent d'insérer plus facilement des valeurs de variables au sein d'une chaîne de caractère, ces dernières peuvent également être définies sur plusieurs lignes.

```
let jedi = {surname:"Obiwan", name:"Kenobi"};
let msg = `${jedi.name} ${jedi.surname} is the Jedi Master`;
console.log(msg);
```

Paramètres par défaut

Il est dorénavant possible de donner une valeur par défaut aux paramètres, cela évite les lignes de codes superflues et facilite la lecture du code.

```
function whoIsTheSithLord(p_name='Dark Vador')
{
    console.log("The Sith lord is: ", p_name);
}

whoIsTheSithLord();
```

Les fonctions fléchées/lambdas

Il s'agit d'une nouvelle façon d'écrire les fonctions anonymes, plus compacte et permettant de lier définitivement le contexte d'éxécution de la fonction (this).

```
function launchFunc(p_function) {
    p_function();
}

// ecma 5 style anonymous function
launchFunc(
    function () {
        console.log("I am The Sith Lord")
    }
);

// ecma6 arrow function
launchFunc(
    () => { console.log("I am The Sith Lord") }
);
```

Les promesses

L'objet Promise (pour « promesse ») est utilisé pour réaliser des traitements de façon asynchrone. Une promesse représente une valeur qui peut être disponible maintenant, dans le futur voire jamais.

L'interface Promise représente un intermédiaire (proxy) vers une valeur qui n'est pas nécessairement connue au moment de sa création. Cela permet d'associer des gestionnaires au succès éventuel d'une action asynchrone et à la raison d'une erreur. Ainsi, des méthodes asynchrones renvoient des valeurs comme les méthodes synchrones, la seule différence est que la valeur retournée par la méthode asynchrone est une promesse (d'avoir une valeur plus tard).

```
new Promise((resolve, reject) => {
    const xhr = new XmlHttpRequest();
    xhr.open("GET", url);
    xhr.onload = () => resolve(xhr.responseText);
    xhr.onerror = () => reject(xhr.statusText);
});

function resolve(p_msg){
    console.log("resolve: ", p_msg);
}

function reject(p_msg){
    console.log("reject: ", p_msg);
}
```

Classes et Héritages

Ecma 6 introduit des mécanismes Orienté Objet par classe grâce aux mots-clés **class** et **extends**, malheureusement ces derniers n'introduisent aucun changement dans la manière dont les objets sont réellements agencés et conçus au sein de la VM Javascript, il s'agit uniquement de sucre syntaxique.

```
class Personnage {
    constructor() {
        this.name = "";
    }

    sayMyName() {
        console.log(this.name);
    }
}

class JediKnight extends Personnage {
    constructor(p_name) {
        this.power = 250;
        this.name = p_name;
    }
}

let obiwan = new JediKnight("Obiwan Kenobi");
obiwan.sayMyName();
```

Les Observables

Un Observable est un producteur de données qui peut être observé. On le mettra sous observation avec la méthode **subscribe** et cette observation sera exécutée par un objet de type Observer. Les Observables se sont imposés dans le monde de la programmation réactive (reactive programming) par le biais de la librairie Reactive X, abrégé RxJs pour sa version Javascript. Un Observable peut également produire des données de façon asynchrone et il est possible de fusionner plusieurs Observables entre eux.

Il faut bien noter que les Observables sont souvent associés aux promesses, voire décrits comme des super-promesses, ce qui n'est pas tout à fait exact, un article entier a d'ailleurs été rédigé à l'attention des développeurs à l'adresse suivante: https://medium.com/@benlesh/learning-observable-by-building-observable-d5da57405d87 et explique un peu plus en détail ce que sont les Observables.

```
Rx.Observable.create(function(observer) {
    setTimeout(() => observer.next("valeur A"), 700);
    setTimeout(() => observer.next("valeur B"), 400);
})
 .subscribe(e => console.log(e));

 // affiche "valeur A", puis "valeur B"  dans la console
```

Reactive Extension For Javascript (RxJS)

Reactive Extensions (Rx) est une bilbiothèque permettant de développer des applications "orientées évènements" et utilisant des requêtes asynchrones en liant entre elles des séquences d'objets de type Observables.

Les séquences de données peuvent prendre plusieurs formes, telle qu'un flux de données issue d'un webservice, des requêtes vers des webservices, des notifications systèmes, ou bien encore une séquence d'évènements utilisateurs etc...

Rx represente toutes ces données sous la forme d'une séquence d'Observables, une application peut souscrire à ce flux de données et être notifiée chaque fois qu'une donnée est ajoutée au flux.

La comparaison avec les promesses est alors tentante et il est possible de se faire une idée plus précise de ce que sont les Observables et dans quels domaine les préférer par rapport aux Promises à l'adresse suivante: https://xgrommx.github.io/rx-book/index.html

Les outils

L'une des choses les plus difficiles à déterminer lorsque l'on commence un développement en Angular est la stack complète de développement. Fort heureusement, une multitude d'outils tous plus performants les uns que les autres sont venus renforcer l'écocsystème Javascript ces derniers temps, et ce, grâce à **node.js** et son gestionnaire de dépendances spécialisé front-end **npm**. L'on peut donc compter au rang des outils quasiment indispensables:

- Node.Js et NPM (node package manager)
- Typescript (langage poussé par Google pour le développement Angular)
- Un bon éditeur Typescript (Micorsoft Visual Code)
- Un outil de scaffolding (génération de structure de projets et codes) comme @angular/cli
- Un module loader, comme Webpack
- Un bon navigateur et/ou environnement d'éxécution (Google Chrome, Ionic ...)

Le langage Typescript

TypeScript est un langage de programmation libre et open source développé par Microsoft qui a pour but d'améliorer et de sécuriser la production de code JavaScript. C'est un sur-ensemble de JavaScript (c'est-à-dire que tout code JavaScript correct peut être utilisé avec TypeScript). Le code TypeScript est transcompilé en JavaScript, pouvant ainsi être interprété par n'importe quel navigateur web ou moteur JavaScript.

TypeScript permet un typage statique optionnel des variables et des fonctions, la création de classes et d'interfaces, l'import de modules, tout en conservant l'approche non-contraignante de JavaScript. Il supporte la spécification ECMAScript 6.

- Typage statique
- Typage générique
- Interfaces
- Classe, classe abstraite, expressions de classe
- Modules
- Mixin
- Enumérations
- Paramètres optionnels
- Symboles

... Et bien d'autres, d'ailleurs depuis la version 1.6, le format JSX est supporté.

ES6 Module Pattern

La norme ES6 prévoit de founir au développeur la possibilité de regrouper et empaqueter des objets, classes, fonctionnalités et valeurs au sein de **modules**, dans la continuité des pattern modulaires déjà bien ancrés au sein de l'écosystème Javascript (AMD par exemple).

L'instruction export est utilisée pour permettre d'exporter des fonctions et objets ou des valeurs primitives à partir d'un fichier (ou module) donné. Ces fonctions et objets peuvent ensuite être utilisées dans d'autres fichiers grâce à import.

Cette fonctionnalité n'est pas encore implémentée nativement par la plupart des navigateurs, en revanche, elle est gérée par de nombreux transpileurs comme **Traceur, Babel ou Rollup.**

```
function whoIsTheSithLord() {
    console.log("Palpatine is the Sith Lord !");
}

// on exporte une fonction déclarée plus haut
export { whoIsTheSithLord };

// on exporte une constante
export const DARK_VADOR = "Dark Vador";
```

ES7 Async functions

La déclaration async function définit une fonction asynchrone qui renvoie un objet AsyncFunction. On peut également définir des fonctions asynchrones grâce au constructeur AsyncFunction et via une expression de fonction asynchrone.

Lorsqu'une fonction asynchrone est appelée, elle renvoie une promesse. Lorsque la fonction asynchrone renvoie une valeur, la promesse est résolue avec la valeur renvoyée. Lorsque la fonction asynchrone lève une exception, la promesse est rompue avec la valeur de l'exception.

Une fonction asynchrone peut contenir une expression await qui permet d'interrompre l'exécution de la fonction asynchrone en attendant la résolution d'une promesse passée à l'expression. L'exécution de la fonction asynchrone reprend lorsque la promesse est résolue.

L'objectif des fonctions asynchrones avec await est de simplifier le comportement des promesses lors d'opérations synchrones et d'opérer sur des groupes de promesses. Si les promesses sont en quelque sorte des callbacks organisés, async/await permet de combiner les générateurs et les promesses. On peut trouver des exemples à l'adresse suivante: https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Instructions/async_function

Module Loader & Webpack

Un module loader est un outil qui fournit une solution permettant de gérer des modules Javascript, en respectant la norme des ES6 Modules. Il s'agit en règle générale d'une suite logicielle composée d'un utilitaire en ligne de commande et de fichiers à configurer.

L'un des plus célèbres Module Loader de l'écosystème Javascript est Webpack, ce dernier permet, comme son nom l'indique, de créer des paquetages à partir de fichier Javascript (mais pas que) et de les mettre à disposition sous forme de modules au sein d'une application front-end.

Pour plus de renseignements sur le fonctionnement de Webpack, rendez-vous sur le site officiel du produit à l'adresse suivante: https://webpack.github.io/docs/

LA PROGRAMMATION EVENEMENTIELLE

Javascript, un langage évènementiel

Javascript fait partie de la famille des langages évènementiels comme Actionscript, Lingo, Qt en C++ (bibliothèque), cela signifie qu'il nous procure, de manière native (ou à l'aide de bibliothèques comme jQuery par exemple), des mécanismes pour programmer de façon évènementielle.

En informatique, la programmation événementielle s'oppose à la programmation séquentielle. Le programme sera principalement défini par ses réactions aux différents événements qui peuvent se produire, çàd des changements d'état de variable, l'incrémentation d'une liste, un mouvement de souris ou de clavier...

En Javascript, elle est surtout utilisée avec les objets du DOM, mais d'autres objets peuvent être utilisés en programmation évènementielle. Pour cela, il faut utiliser conjointement les méthodes suivantes des objets implémentant l'interface IEventDispatcher:

- **addEventListener**: Qui nous permet d'ajouter un écouteur d'évenements (une fonction de rappel) qui se déclenchera en réaction à un évènement.
- **removeEventListener**: Qui nous permet d'enlever un écouteur d'évènements ajouté auparavant.
- **dispatchEvent**: Qui nous permet de lancer un évènement à la main, s'utilise avec les objets de type Event
- **Objet Event**: Cet objet est utilisé pour créer des évènements déjà connus ou personnalisés, attention les évènements utilisateurs tels que les clicks de souris, les frappes au clavier ne peuvent pas être simulés parfois, pour des raisons de sécurité.

Les évènements de chargement de page

En Javascript, il est courant d'attendre que la page soit chargée avant de déclencher un script quelconque. pour cela, il existe plusieurs méthodes, notamment une qui nous dicte d'inclure le script en tant que dernier enfant du noeud body.

Parfois, cela peut être une bonne méthode, mais le plus souvent, il s'agit d'une mauvaise pratique. En effet, il existe deux évènements spécifiques au chargement d'une page, l'un envoyé par l'objet window, et l'autre par l'objet document:

- **load event**: Envoyé par l'objet window lorsque l'ensemble des noeuds HTML de la page et toutes les ressources qui y sont associées (images, vidéos, sons etc...) sont parsés, téléchargés, prêts à être manipulés.
- **DOMContentLoaded event**: Envoyé par l'objet window.document lorsque l'ensemble du HTML de la page est parsé et prêt à être manipulé. On ne tient pas compte des autres ressources.

```
function onPageFullyLoaded(event){
    console.log("HTML & other ressources fully loaded");
}

function onHTMLFullyLoaded(event){
    console.log("HTML fully loaded");
}

window.addEventListener("load", onPageFullyLoaded, false );
window.document.addEventListener("DOMContentLoaded", onHTMLFullyLoaded, false);
```

Les évènements utilisateur

Il est possible de détecter des évènements déclenchés par une action de l'utilisateur comme par exemple:

- Le click souris: à l'aide de l'évènement click'
- Passer par dessus un élément: à l'aide de l'évènement mouseover
- Appuyer sur une touche du clavier: à l'aide de l'évènement keydown

```
function keyDownHandler(event){
    console.log("appui sur une touche");
}
window.addEventListener("keydown", keyDownHandler, true);


function clickHandler(event){
    console.log("click souris");
}

window.addEventListener("click", clickHandler, true);
```

Il en existe beaucoup d'autres que vous pouvez découvrir sur la documentation officielle Javascript de Mozilla Developer Network: https://developer.mozilla.org/fr/

Différences entre les navigateurs (non modernes)

Il existe des différences d'API entre les navigateurs les plus anciens en ce qui concerne la programmation évènementielle. Par exemple, au lieu de la méthode standardisée **addEventListener** le navigateur Internet Explorer a très longtemps utilisé la méthode **attachEvent** ainsi que les méthodes ** detachEvent et fireEvent**.

Mais il n'y a pas qu'Internet Explorer qui pouvait poser problème, à l'époque chaque navigateur avait ses propres subtilités et il était assez compliqué de programmer efficacement de façon évènementielle. C'est en partie pour répondre à ce besoin qu'a été créée la bibliothèque **jQuery**.

NOTIONS AVANCEES

Les valeurs truthy/falsy

Si une valeur peut être convertie en true, on dit en anglais qu'elle est truthy. Pour false on dit qu'elle est falsy. Parmi les expressions qui peuvent être converties en false, citons celles qui seront évaluées à :

- null
- 0
- la chaîne vide ("")
- undefined.

Même si les opérateurs && et || peuvent être utilisés avec des opérandes qui ne sont pas des valeurs booléennes, ils peuvent toujours être considérés comme des opérateurs booléens puisque leurs valeurs de retour peuvent toujours être converties en valeurs booléennes.

Comme les expressions logiques sont évaluées de gauche à droite, leur évaluation sera éventuellement « court-circuitée » à l'aide des règles suivantes :

l'évaluation de false &&n'importe quoi est court-circuitée en false.

l'évaluation de true ||n'importe quoi est court-circuitée en true.

Les règles de la logique garantissent que ces évaluations seront toujours correctes. Notons que la partie n'importe quoi des expressions mentionnées ci-dessus ne sera jamais évaluée, et que tout effet de bord éventuel induit par cette évaluation ne se produira pas.

L'opérateur in

L'opérateur in renvoie true si la propriété donnée appartient à l'objet donné et s'utilise avec la syntaxe suivante: **ma_propriete in nomObjet**. Ici, 'ma_propriete' est une expression évaluée en un nombre ou une chaîne de caractères qui représente le nom d'une propriété ou l'indice d'un tableau et 'nomObjet' représente l'objet que l'on souhaite inspecter.

```
var tab = ["New York"];

// affiche true
console.log(0 in tab);

// affiche false car il n'existe pas d'éléments à l'indice 1 dans le tableau
console.log(1 in tab);

// affiche true, length est une propriété des objets de type tableau
console.log("length" in tab)

// on peut utiliser l'opérateur in dans une boucle for .. in
var obj = { "age": 40, "name": "Dark Vador" }, prop = null;
for (prop in obj) {

   // affiche tour à tour "age" et "name"
   console.log(prop);
}
```

L'opérateur delete

L'opérateur delete permet de retirer une propriété donnée d'un objet. Lorsque la suppression se déroule sans problème, l'opération renvoie true, sinon c'est la valeur false qui est renvoyée. Voici quelques scénarios importants qui précisent ce comportement :

- Si une propriété du même nom existe sur la chaîne de prototypes ne peuvent être supprimées
- Toute propriété déclarée avec var ne peut pas être supprimée de la portée globale ou de la portée d'une fonction.
- Les propriétés propriétés des objets natifs comme Array ne peuvent être supprimées.

```
var obj = {nom: "Dark Vador"};

// la propriété est supprimée, renvoie true
delete obj.nom;

// renvoie false et lève une erreur
delete Array.splice;
```

Conversions de valeurs

Il existe 2 façons de convertir des valeurs en Javascript, la première consiste à convertir soi-même les données, la seconde consiste à laisser Javascript opérer ce que l'on appelle une **conversion implicite.**

Pour les conversions explicites, faites à la main, il existe un panel de fonctions telles que parseInt, parseFloat, les types de données eux-mêmes (String) (Number) (Date) etc ...

Mais il existe des cas ou Javascript va chercher à convertir une donnée dans un autre type de donnée, essentiellement lors des comparaisons non strictes de données. Dans cette situation **La donnée du type le plus complexe va être convertie dans le type de données le plus simple.**

```
var age = "30.5";

// convertit la chaîne en nombre flottant
console.log(parseFloat(age));

// convertit la chaîne en nombre entier
console.log(parseInt(age));

// conversion implicite de données, le type le plus complexe (String)
// va être automatiquement converti en (Number) par Javascript
console.log( age == 30.5 );
```

Egalité stricte

Il existe 2 mécanismes de comparaisons en Javascript, la comparaison stricte et la comparaison non stricte. La comparaison non stricte est la plus employée car elle utilise l'opérateur **==**. Cet opérateur va tenter de comparer les valeurs contenues dans les 2 opérandes mais ne va pas se soucier du type de données desdits opérandes.

La comparaison stricte utilise l'opérateur **===**, cet opérateur va comparer les valeurs des 2 opérandes ainsi que les types de données. Ainsi, aucune opération de conversion implicite ne sera effectuée lors de la comparaison.

```
// renvoit true, seules les valeurs sont comparées.
console.log("10" == 10);

// renvoit false, valeurs & types comparées
console.log("10" === 10);
```

Programmation fonctionnelle

La programmation fonctionnelle est une approche de la programmation différente de l'approche orientée objet, à laquelle elle peut accessoirement se combiner. Elle produit des abstractions de langage utiles en combinant astucieusement des fonctions, de sorte à ce que le code soit plus facile à lire.

Par exemple, l'itération au sein d'une liste peut être décrite comme la combinaison de 2 actions, l'une nous permet de boucler sur la liste, la seconde nous permet d'effectuer une action sur l'élement en cours. Dans l'exemple ci-dessous, on réduit un tableau à la somme de tout ces élements:

```
function reduce(action, tab, base) {
    for (var i = 0; i < tab.length; i++) {
        base = action(base, tab[i]);
    }
    return base;
}

function add(a, b){
    return a + b;
}

// nous retourne la somme des élements du tableau
console.log( reduce(add, [0,30,80,100], 0 ));
```

Masquage et portée des variables

En Javascript, une variable est accessible dans l'environnement dans lequel elle a été définie, çàd que si la variable est définie dans l'espace global (window), alors elle sera accessible dans tout les environnements enfants de window telles que des fonctions etc ...

En fait, à chaque fois qu'une variable est déclarée dans un environnement (par exemple une fonction), elle devient automatiquement une propriété de cet environnement, puisque tout ou presque est objet en Javascript.

```
// cette variable devient automatiquement une propriété de l'environnement
// dans lequel elle est définie (window) et est donc disponible dans tout
// les environnements "enfants"
var sith_lord = "Dark Vador";

function whoIsTheSithLord(){
    // affiche Dark Vador dans la console
    console.log(sith_lord);
}

whoIsTheSithLord();
```

Les closures

Une fermeture, ou closure en anglais, est une fonction qui fait utiliser des variables indépendantes (utilisées localement mais définies dans la portée englobante). Autrement dit, ces fonctions se « souviennent » de l'environnement dans lequel elles ont été créées (on dit aussi que la fonction capture son « environnement »).

Normalement, les variables locales d'une fonction n'existent que pendant l'exécution d'une fonction cependant, en Javascript, une fonction se souvient toujours de son contexte interne, çàd des variables et autres fonctions créées en son sein, elles fonctionnent donc comme des objets (car ce sont des objets).

```
function getSithLordFunc() {

    var sith_lord = "Dark Vador";
    function whoIsTheSithLord() {
        console.log(sith_lord);
    }

    return whoIsTheSithLord;
}
// retourne la fonction whoIsTheSithLord...
var func = getSithLordFunc();

// invoque la fonction whoIsTheSithLord qui se souvient
// de la variable sith_lord décrite dans le même contexte
func();
```

Comprendre le hoisting (hissage)

En JavaScript, functions et variables sont hissées ('hoisting' est traduit en français par 'hissage'). Le hissage est un comportement du Javascript qui déplace une déclaration au début du scope dans lequel il est déclaré (scope courant de la fonction ou dans le global scope). Cela signifie qu'il est possible d'utiliser une fonction ou une variable avant qu'elle ne soit déclarée. Autrement dit, une fonction ou variable peut-être déclarée après son utilisation.

```
// affiche Dark Vador dans la console
console.log(sith_lord);

var sith_lord = "Dark Vador";

// affiche Dark Vador dans la console
whoIsTheSithLord();

function whoIsTheSithLord() {
    console.log("Dark Vador");
}
```

Le Module Pattern (AMD)

En Javascript, il existe un design pattern nommée 'module pattern' (abrégé amd), qui permet d'empaqueter des objets, fonctions et variables à l'intérieur d'un objet sobrement nommée 'module' crée à l'aide d'une fonction qui s'invoque elle-même et qui retourne un résultat, un objet en règle générale. Le développeur va donc pouvoir exporter, via les propriétés de l'objet retourné, les méthodes et propriétés qu'il souhaite exposer à l'extérieur du module et garder 'privées' toutes les autres.

```
var module = (
    function () {

        function _privateMethod() {
            console.log("private method");
        }

        function publicMethod() {
            _privateMethod();
            console.log("public method");
        }

        return {
            myPublicMethod: publicMethod
        }

    }
)();

module.myPublicMethod();
```

Apply & call

La méthode apply() des objets de type function, invoque la fonction en lui passant une valeur this et des arguments sous forme d'un tableau (ou d'un objet semblable à un tableau). Ainsi le développeur peut s'assurer du contexte dans lequel est invoquée la fonction lors de l'appel à celle-ci.

La méthode call est similaire à apply(), la différence fondamentale réside dans le fait que call() accepte une liste d'arguments, tandis que la méthde apply() accepte un unique tableau d'arguments.

```
function wholsTheSithLord() {
    console.log(this.name);
}

var sith_lord = {
    name: "Dark Vador"
};

// invoque la fonction, l'opérateur this pointera vers l'objet sith_lord
wholsTheSithLord.apply(sith_lord);
```

Garantir le contexte (binding)

La fonction bind() crée une nouvelle fonction qui, lorsqu'elle est appelée, a pour contexte this la valeur passée en paramètre et éventuellement une suite d'arguments qui précéderont ceux fournis à l'appel de la fonction créée. Cela permet au développeur de garantir le contexte d'éxécution d'une fonction et de résoudre ce type de problème:

```
var sith_lord = {
    name: "Dark Vador",
    whoIsTheSithLord: function () {
        console.log(this.name);
    }
};

// affiche Dark Vador
sith_lord.whoIsTheSithLord();

// affiche undefined, this vaut window ici...
var func = sith_lord.whoIsTheSithLord;
func();

// affiche Dark Vador car le contexte (this) est lié à sith_lord
func = sith_lord.whoIsTheSithLord.bind(sith_lord);
func();
```

La console de debug

Il existe une multitude d'outils pour nous aider à débugger en Javascript mais le plus complet (et donc le plus utile) est dans aucun doute la console du navigateur. En effet celle-ci nous permet de poser des points d'arrêts (breakpoints), des watchers (watch), de vérifier ce qui se passe sur le réseau (panneau network) etc etc ... Voici une liste des panneaux utiles que vous pourrez trouver dans la console de Google Chrome par exemple:

- Le panneau sources
- Le panneau ressources
- Le panneau network
- Le panneau timeline
- Le panneau profiles
- Le panneau audits

Web Workers, Service Workers

Les Web Workers fournissent un moyen simple d'exécuter des scripts dans des threads en arrière-plan d'un contenu web. Une fois créé, un worker peut envoyer des messages à son thread parent. Un worker dédié est seulement accessible à partir du script qui l'a créé, tandis que les workers partagés peuvent être interrogés par de multiples scripts.

Les workers s'exécutent dans un un contexte global différent du contexte de la fenêtre courante, utiliser window pour obtenir le contexte global courant (au lieu de self) au sein d'un Worker retournera une erreur.

```
//script principal
var sith_lord = new Worker("dark_vador.js");
sith_lord.onmessage = function (event) {
    // affiche Dark Vador
    console.log(event.data);
};

sith_lord.postMessage("getsithlord");


// dark_vador.js
onmessage = function (event) {

    if (event.data == "getsithlord")
        postMessage("Dark Vador");
}
```

Les tests unitaires

Les tests unitaires sont une depuis bien longtemps, une norme dans le monde du développement Backend, toutefois on peut dire qu'au sein de l'écosystème Javascript, l'écriture et l'utilisation de tels tests n'est pas entré tout de suite dans les moeurs.

Il existe cependant depuis plus de 10 ans, des frameworks de tests unitaires disponibles pour ce langage et aujourd'hui certains rivalisent d'inventivité afin de procurer au développeur toujours plus de flexiblité et d'efficacité dans sa démarche de test.

Voici une petite liste (non exhaustive) des différents frameworks de tests unitaires les plus utilisés au moment ou ces lignes sont écrites:

- Jasmine: https://jasmine.github.io/
- Mocha: https://mochajs.org/
- Karma: https://karma-runner.github.io/1.0/index.html
- UnitJS: http://unitjs.com/

ES6

ECMAScript 2015 (sixième édition) est la version actuelle pour le langage ECMAScript. Anciennement appelé « ES6 », c'est le standard qui définit l'implémentation de JavaScript, notamment pour SpiderMonkey, le moteur utilisé par Firefox et les autres applications Mozilla.

Javascript propose donc une version récente du langage qui remplit les spécifications de cette norme, qui apporte son lot de nouveautés notamment:

- Des classes
- Un système d'héritage
- Un système de module intégré
- Le suppport des lambdas functions
- Etc...

Utiliser ES6 aujourd'hui

ES6 est une norme récente, par conséquent, il n'est pas possible de l'utiliser avec tout les navigateurs. En effet, les moins récents ne proposent pas de support pour cette version du langage Javascript. Afin de pouvoir l'utiliser partout, il nous faudra passer notre code ES6 à un outil que l'on appelle un 'transpiler', tel que Babel. Cet outil aura la charge de convertir notre code ES6 en sa version ES5.

Pour plus de détails sur le fonctionnement de Babel, vous pouvez vous rendre sur le site https://babeljs.io/ et commencer dès maintenant à coder en ES6. S'y mettre de suite reste une bonne idée, car le standard est maintenant finalisé et ne bougera probablement plus à l'avenir (ou très peu), une fois que tout les navigateurs le supporteront, ES5 disparaîtra et vous devrez quoiqu'il arrive vous mettre à l'ES6, autant prendre de l'avance.

React, fonctionnalités avancées du framework de développement front-end

Introduction aux fonctionnalités avancées de React

Cette formation part du principe que les bases de React (JSX, composants, props) sont déjà connues. On se concentre désormais sur ce qui fait la différence en production : cycle de vie, refs, composition et hooks.

Tout au long du cours, nous ferons évoluer **WorldGuide**, une application de guide de pays. Chaque chapitre ajoute une brique réelle : état local, cache, navigation, tests, puis déploiement.

L'objectif n'est pas d'empiler des APIs, mais de savoir **quand** les utiliser, quels pièges elles introduisent, et comment les assembler dans une architecture maintenable.

- **Composants** : classes historiques vs fonctions + hooks
- **Cycle de vie** : montage, mise à jour, démontage et effets
- **APIs intermédiaires** : refs, fragments, PropTypes / TypeScript
- **Fil rouge WorldGuide** : liste de pays, fiche détail, favoris

```jsx
// Objectif : un composant fonctionnel simple (liste de pays en dur)

import { useState } from 'react';

const COUNTRIES = [
  { code: 'FR', name: 'France', capital: 'Paris' },
  { code: 'JP', name: 'Japon', capital: 'Tokyo' },
  { code: 'BR', name: 'Brésil', capital: 'Brasília' },
];

export function CountryList({ countries }) {
  return (
    <ul>
      {countries.map((country) => (
        <li key={country.code}>
          {country.name} — {country.capital}
        </li>
      ))}
    </ul>
  );
}

export default function WorldGuideIntro() {
  const [countries] = useState(COUNTRIES);

  return (
    <main>
      <CountryList countries={countries} />
    </main>
  );
}
```

Cycle de vie des composants

Un composant React passe par trois grandes phases : .

- **Montage** : initialiser l'état, s'abonner, lancer le premier chargement (**componentDidMount**)
- **Mise à jour** : réagir aux props / state, éviter les boucles infinies (**componentDidUpdate**)
- **Démontage** : abort des requêtes, clearInterval, désabonnement (retrait de l'arbre) **componentWillUnmount**
- **Règle** : ne jamais déclencher d'effets de bord pendant le rendu

```
// cycle de vie : chargement, mise à jour, nettoyage
import { Component, useEffect, useState } from 'react';

const API = 'MY_URL_WITH_COUNTRIES';

/*version classe*/
export class CountryLoaderClass extends Component {
  state = { countries: [], error: null };

  async componentDidMount() {
    try {
      const response = await fetch(API);
      const data = await response.json();
      this.setState({ countries: data });
    } catch (error) {
      this.setState({ error: error.message });
    }
  }

  componentWillUnmount() { /* abort fetch if necessary*/ }

  render() {
    const { countries, error } = this.state;
    if (error) return <p>Erreur : {error}</p>;
    return <p>{countries.length} pays chargés (classe)</p>;
  }
}
```

```jsx
// cycle de vie : chargement, mise à jour, nettoyage
import { Component, useEffect, useState } from 'react';

const API = 'MY_URL_WITH_COUNTRIES';

/*version fonction*/
export function CountryLoader({ region }) {
  const [countries, setCountries] = useState([]);

  useEffect(() => {
    async function load() {
      const response = await fetch(API+"?region="+region);
      setCountries(await response.json());
    }

    load();
    return () => {/* abort fetch if necessary*/};
  }, [region]);

  return <p>{countries.length} pays — région : {region}</p>;
}
```

Refs, fragments et PropTypes

- Les **Fragments <>...</>** ou **<React.Fragment>** permettent de grouper des enfants sans noeud DOM superflu..
- **useRef** : DOM (input de recherche) ou valeur mutable (id de timer)
- **forwardRef** : exposer un focus depuis un champ encapsulé
- Les **refs** donnent un accès à un noeud DOM ou à une valeur persistante entre les rendus, sans provoquer de re-rendu. On les réserve au focus, aux mesures, aux animations ou à l'intégration de bibliothèques non React.
- **PropTypes** documente le contrat d'un composant à l'exécution. TypeScript remplace souvent ce rôle, mais cela reste utile pour des projets JS pures.

```
// refs (focus recherche), fragments, PropTypes
import { forwardRef } from 'react';
import PropTypes from 'prop-types';

export const SearchField = forwardRef(function SearchField({ value, onChange }, ref) {
  return (
    <input  ref={ref} value={value}
          onChange={(event) => onChange(event.target.value)}/>
  );
});

SearchField.propTypes = {
  value: PropTypes.string.isRequired,onChange: PropTypes.func.isRequired
};
```

```jsx
// refs (focus recherche), fragments, PropTypes
import { Fragment } from 'react';
import PropTypes from 'prop-types';



export function CountryRow({ country }) {
  return (
    <Fragment><td>{country.name}</td><td>{country.capital}</td></Fragment>
  );
}

CountryRow.propTypes = {
  country: PropTypes.shape(
    {name: PropTypes.string.isRequired,capital: PropTypes.string }
  ).isRequired,
};



// refs (focus recherche), fragments, PropTypes
import { useRef } from 'react';


export default function CountrySearch({ countries, query, onQueryChange }) {
  const inputRef = useRef(null);

  return (
    <div>
      <button type="button" onClick={() => inputRef.current?.focus()}>
        Recherche
      </button>
      <SearchField ref={inputRef} value={query} onChange={onQueryChange} />
      <table>
          {countries.map((country) => (
            <tr key={country.code}>
              <CountryRow country={country} />
            </tr>
          ))}
      </table>
    </div>
  );
}
```

React Hooks

Les **hooks** permettent d'ajouter de l'état, des effets et de la réutilisation de logique à des composants fonctions. Ils remplacent, dans la pratique, la grande majorité des classes.

Les règles sont strictes : appeler les hooks uniquement au **plus haut niveau** du composant (jamais dans un if / boucle) et uniquement depuis un composant React ou un hook personnalisé. L'ordre des appels doit rester stable d'un rendu à l'autre.

```
import { useState } from 'react';

// Hook perso
function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    try {
      return JSON.parse(localStorage.getItem(key)) ?? initial;
    } catch {
      return initial;
    }
  });

  const set = (v) => {
    setValue(v);
    localStorage.setItem(key, JSON.stringify(v));
  };

  return [value, set];
}
```

```jsx
import { useState, useEffect, useContext, useMemo, useCallback, createContext } from 'react';

const ThemeCtx = createContext('light');

export default function App() {
  const [theme, setTheme] = useState('light');
  const [count, setCount] = useState(0);
  const [n, setN] = useState(1);
  const [name, setName] = useLocalStorage('name', 'Alice');

  useEffect(() => { document.title = `Count: ${count}`; }, [count]);

  const costly = useMemo(() => {
    let r = 0;
    for (let i = 0; i < 1e6; i++) r += n * i;
    return r;
  }, [n]);

  const add = useCallback(() => setCount(c => c + 1), []);

  return (
    <ThemeCtx.Provider value={theme}>
      <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>
        Thème: {theme}
      </button>

      <p>{count}</p>
      <button onClick={add}>+1 (useCallback)</button>

      <Btn />

      <p>n={n} → {costly}</p>
      <button onClick={() => setN(n + 1)}>n+1 (useMemo)</button>

      <input value={name} onChange={e => setName(e.target.value)} />
      <p>{name}</p>
    </ThemeCtx.Provider>
  );
}

function Btn() {
  const theme = useContext(ThemeCtx);
  return <button style={{ background: theme === 'dark' ? '#333' : '#ddd' }}>Bouton ({theme})
</button>;
}
```

Introduction à React et l'écosystème JavaScript

Écosystème JavaScript moderne et ses outils

Un projet React s'appuie sur une chaîne d'outils : **Node.js** et npm/pnpm/yarn pour les dépendances, un **bundler** (Vite, Webpack, Turbopack) pour transformer JSX et modules, et TypeScript pour typer le contrat des composants.

Vite est le choix le plus fréquent aujourd'hui pour une SPA : serveur de dev ultra-rapide, HMR, build Rollup. Create React App est en fin de vie ; Next.js entre en jeu dès qu'on a besoin de SSR, de routes fichiers ou de Server Components.

Pour WorldGuide, on initialise un projet Vite + React, on active ESLint / Prettier, et on prépare les scripts **dev**, **build** et **preview**.

- **Runtime** : Node LTS, gestionnaire de paquets, lockfile versionné
- **Build** : Vite (SPA) ou Next.js (SSR / RSC)
- **Qualité** : ESLint, Prettier, TypeScript, tests (Vitest / Jest)
- **Git** : branches, hooks pre-commit (lint-staged)

```
// Chaîne d'outils typique d'un projet Vite + React (WorldGuide)
export const scripts = {
  dev: 'vite',
  build: 'vite build',
  preview: 'vite preview',
  lint: 'eslint src --ext js,jsx',
  test: 'vitest',
};

export const recommendedStack = {
  bundler: 'Vite',
  language: 'JavaScript ou TypeScript',
  linter: 'ESLint + plugin react-hooks',
  formatter: 'Prettier',
  testRunner: 'Vitest + Testing Library',
};

export function printBootstrapHint() {
  return [
    'npm create vite@latest worldguide -- --template react',
    'cd worldguide && npm install',
    'npm run dev',
  ].join('\n');
}
```

Principes fondamentaux de React et son architecture

Dans React on ne manipule pas le DOM à la main ; on déclare ce qui doit s'afficher, et React calcule le différentiel.

L'architecture repose sur un arbre de composants, un flux de données **unidirectionnel** (props descendantes, événements remontants) et un moteur de réconciliation (Fiber) capable d'interrompre un rendu pour rester fluide.

- **Déclaratif** : décrire le résultat, pas les étapes DOM
- **Unidirectionnel** : props down, events up
- **Composition** : petits composants plutôt qu'un monolithe
- **Immuabilité** : nouvel état, pas de mutation silencieuse

```
// UI = f(state) — flux unidirectionnel WorldGuide
export function reduceWorldGuide(state, action) {
  switch (action.type) {
    case 'setQuery':
      return { ...state, query: action.query };
    case 'selectCountry':
      return { ...state, selectedCode: action.code };
    default:
      return state;
  }
}

export function view(state) {
  const visible = state.countries.filter((country) =>
    country.name.toLowerCase().includes(state.query.toLowerCase())
  );
  return { visible, selectedCode: state.selectedCode };
}

export const initialState = {
  query: '',
  selectedCode: null,
  countries: [{ code: 'FR', name: 'France' }],
};
```

JSX : syntaxe, fonctionnement et bonnes pratiques

JSX est du sucre syntaxique compilé en appels **React.createElement** (ou **jsx** runtime automatique). Chaque élément a un type, des props et des enfants.

Les expressions s'écrivent entre accolades. On n'utilise pas **if** comme instruction dans le JSX : on préfère des ternaires, des early returns, ou des variables intermédiaires. Les listes exigent une **key** stable (code ISO, pas l'index si la liste peut être réordonnée).

className, **htmlFor** et les événements **onClick** suivent la convention camelCase. Les attributs booléens s'écrivent **disabled={isLoading}**.

- Une seule racine par return (ou fragment)
- Keys stables sur les listes de pays
- Pas de logique métier lourde dans le JSX : extraire des fonctions
- Éviter **dangerouslySetInnerHTML** (XSS) sauf HTML déjà assaini

```
// Bonnes pratiques JSX — liste de pays
export function CountryList({ countries, region }) {
  if (!countries.length) {
    return <p>Aucun pays à afficher.</p>;
  }

  return (
    <>
      <h2>{region ? `Région ${region}` : 'Tous les pays'}</h2>
      <ul>
        {countries.map((country) => (
          <li key={country.code}>
            {country.flag} {country.name}
          </li>
        ))}
      </ul>
    </>
  );
}

export function FlagOrFallback({ flag, name }) {
  return flag ? <span aria-hidden="true">{flag}</span> : <span>{name[0]}</span>;
}
```

Virtual DOM et mécanismes de réconciliation

Le **Virtual DOM** est un arbre JavaScript léger qui décrit l'UI. Après un setState, React compare (diff) le nouvel arbre à l'ancien et n'applique au DOM réel que les mutations nécessaires.

L'heuristique principale : si le **type** d'un nœud change (**div** → **section**, ou un autre composant), React démonte tout le sous-arbre. Les **keys** permettent d'identifier des frères (liste de pays) pour réutiliser les instances au lieu de les recréer.

Fiber (React 16+) découpe le travail en unités interruptibles. En React 18, le rendu concurrent peut préparer une UI en arrière-plan (transitions) sans bloquer les saisies dans le champ de recherche WorldGuide.

- Même type + même key → mise à jour des props
- Type différent → unmount / remount (état local perdu)
- Index comme key → bugs au tri / suppression de pays
- React.memo / shouldComponentUpdate : court-circuiter le diff

```
// Réconciliation : la key décide si l'état local survit
import { useState } from 'react';

function CountryNote({ name }) {
  const [note, setNote] = useState('');
  return (
    <label>
      Note sur {name}
      <input value={note} onChange={(e) => setNote(e.target.value)} />
    </label>
  );
}

export function CountryNotesBad({ countries }) {
  return countries.map((country, index) => (
    <CountryNote key={index} name={country.name} />
  ));
}

export function CountryNotesGood({ countries }) {
  return countries.map((country) => (
    <CountryNote key={country.code} name={country.name} />
  ));
}
```

Outils de développement React et debugging

React Developer Tools (extension navigateur) expose l'arbre des composants, les props, le state, les hooks et le Profiler. C'est le premier réflexe quand un CountryCard ne reçoit pas les bonnes données.

En développement, React affiche des warnings utiles (keys manquantes, mises à jour sur un composant démonté). Le Strict Mode double volontairement certains rendus et effets pour révéler les effets non idempotents.

Compléter avec les breakpoints du navigateur, **console** ciblé, et plus tard le Profiler pour les re-renders coûteux de la liste des 250 pays.

- Onglet Components : inspecter props / hooks
- Onglet Profiler : flamegraph des rendus
- Strict Mode : détecter les effets mal nettoyés
- Sourcemaps Vite : déboguer le JSX d'origine

```
// Astuces DevTools : nommer les composants, Strict Mode
import { StrictMode, useEffect, useState } from 'react';

export function CountryCount({ countries }) {
  const [count, setCount] = useState(countries.length);

  useEffect(() => {
    setCount(countries.length);
  }, [countries]);

  return <p data-testid="count">{count} pays</p>;
}

export function DevRoot({ children }) {
  return <StrictMode>{children}</StrictMode>;
}

CountryCount.displayName = 'CountryCount';
```

Composants React fondamentaux

Architecture et anatomie des composants React

Un composant est une fonction (ou une classe) qui reçoit des **props** et retourne du JSX. On distingue les composants **présentationnels** (affichage pur) et les composants **conteneurs** (chargement, état, orchestration).

L'anatomie type d'un fichier : imports, hook / logique, early returns (loading, error, empty), puis le JSX. Extraire dès qu'un bloc a un nom métier clair : **CountryCard**, **RegionFilter**, **FavoriteButton**.

Éviter les composants-dieu de 400 lignes. WorldGuide gagne à séparer page liste, page détail et layout commun.

- Un composant = une responsabilité lisible
- Nommage : fichiers PascalCase, hooks préfixés par use
- Colocation : styles / tests à côté du composant
- Barrel exports avec parcimonie (évite les cycles)

```
// Anatomie : présentation vs conteneur
export function CountryCard({ country, onSelect }) {
  return (
    <article>
      <h3>{country.name}</h3>
      <p>{country.capital}</p>
      <button type="button" onClick={() => onSelect(country.code)}>
        Détail
      </button>
    </article>
  );
}

export function CountryListPage({ countries, onSelect }) {
  if (!countries) return <p>Chargement…</p>;
  if (countries.length === 0) return <p>Aucun résultat</p>;
  return countries.map((country) => (
    <CountryCard key={country.code} country={country} onSelect={onSelect} />
  ));
}
```

Gestion des props et leur validation

Les props sont le contrat public d'un composant : elles sont **en lecture seule**. Toute évolution passe par un callback fourni par le parent (**onSelectCountry**) ou par un état interne.

On destructure les props, on fournit des **valeurs par défaut**, et on évite le props drilling trop profond (voir la section communication). Les children sont une prop particulière, utile pour la composition.

La validation se fait avec TypeScript (préféré) ou PropTypes. Documenter **country**, **isFavorite** et **onToggleFavorite** évite les surprises en équipe.

- Props descendantes, jamais mutées
- Callbacks nommés **onXxx** pour les événements
- Default props / paramètres par défaut
- TypeScript : interfaces **CountryCardProps**

```
import PropTypes from 'prop-types';

export function CountryCard({ country, isFavorite = false, onToggleFavorite }) {
  return (
    <article>
      <h3>{country.name}</h3>
      {isFavorite ? <span>★ favori</span> : null}
      <button type="button" onClick={() => onToggleFavorite(country.code)}>
        {isFavorite ? 'Retirer' : 'Ajouter'}
      </button>
    </article>
  );
}

CountryCard.propTypes = {
  country: PropTypes.shape({
    code: PropTypes.string.isRequired,
    name: PropTypes.string.isRequired,
  }).isRequired,
  isFavorite: PropTypes.bool,
  onToggleFavorite: PropTypes.func.isRequired,
};
```

Patterns de composition de composants

La composition est le modèle de réutilisation de React, plus naturel que l'héritage. Un **PageLayout** reçoit **header**, **children** et **footer** plutôt que d'imposer une hiérarchie de classes.

Les patterns utiles : **children**, slots nommés (props JSX), **compound components** (**Tabs** + **Tabs.Panel**), et plus tard render props / hooks.

Dans WorldGuide, **CountryGrid** compose des **CountryCard** ; **AppShell** compose navigation + zone de contenu.

- Préférer composition à héritage
- Slots : header, sidebar, children
- Compound components pour les API de widgets
- Éviter les HOC sauf interop avec du code ancien

```
export function AppShell({ header, sidebar, children }) {
  return (
    <div className="app-shell">
      <header>{header}</header>
      <div className="app-shell__body">
        <aside>{sidebar}</aside>
        <main>{children}</main>
      </div>
    </div>
  );
}

export function Tabs({ children }) {
  return <div role="tablist">{children}</div>;
}

function Tab({ children, isActive, onSelect }) {
  return (
    <button type="button" role="tab" aria-selected={isActive} onClick={onSelect}>
      {children}
    </button>
  );
}

Tabs.Tab = Tab;
```

Approches de styling modernes

Plusieurs stratégies cohabitent : CSS modules (portées locales), CSS-in-JS (styled-components, Emotion), utilitaires (Tailwind), et feuilles globales pour le reset / le thème.

Le choix dépend de l'équipe. CSS modules + variables CSS restent simples et performants. Tailwind accélère l'UI. Éviter les styles inline pour tout, sauf des valeurs vraiment dynamiques (largeur d'une barre de population).

WorldGuide peut poser un thème clair/sombre via variables CSS et un **data-theme** sur **html**.

- CSS modules : pas de collision de classes
- Tailwind : productivité, design system via config
- CSS variables : thème et densités
- Accessibilité : contraste, focus visible, pas d'info par la couleur seule

```
const themes = {
  light: { bg: '#f7f7f5', fg: '#1a1a1a', accent: '#0b6e4f' },
  dark: { bg: '#121212', fg: '#f5f5f5', accent: '#4ade80' },
};

export function applyTheme(name) {
  const theme = themes[name] ?? themes.light;
  const root = document.documentElement;
  root.style.setProperty('--bg', theme.bg);
  root.style.setProperty('--fg', theme.fg);
  root.style.setProperty('--accent', theme.accent);
  root.dataset.theme = name;
}

export function populationBarStyle(value, max) {
  return { width: `${Math.round((value / max) * 100)}%` };
}
```

Système d’événements React

React enveloppe les événements natifs dans un **SyntheticEvent** : noms camelCase, comportement aligné entre navigateurs, délégation à la racine (root).

Les handlers reçoivent l'événement et, souvent, un identifiant métier : **onSelect(country.code)**. Pour les formulaires, **onChange** + état contrôlé est le modèle par défaut. **preventDefault** sert aux soumissions.

Attention à la performance : ne pas recréer inutilement des closures lourdes sur 250 cartes ; **useCallback** n'est utile que si l'enfant est mémoïsé.

- **onClick**, **onChange**, **onSubmit** : API principale
- État contrôlé vs non contrôlé (refs) pour les champs
- Délégation : un handler sur la liste plutôt que 250 listeners
- Accessibilité : boutons réels, gestion clavier

```jsx
import { useState } from 'react';

export function CountrySearchForm({ onSearch }) {
  const [query, setQuery] = useState('');

  function handleSubmit(event) {
    event.preventDefault();
    onSearch(query.trim());
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={query}
        onChange={(event) => setQuery(event.target.value)}
        name="q"
      />
      <button type="submit">Rechercher</button>
    </form>
  );
}

export function CountryListDelegated({ countries, onSelect }) {
  function handleClick(event) {
    const code = event.target.closest('button')?.dataset.code;
    if (code) onSelect(code);
  }

  return (
    <ul onClick={handleClick}>
      {countries.map((country) => (
        <li key={country.code}>
          <button type="button" data-code={country.code}>
            {country.name}
          </button>
        </li>
      ))}
    </ul>
  );
}
```

État et cycle de vie

Gestion d’état avec useState

useState conserve une valeur entre les rendus et déclenche un nouveau rendu quand on appelle le setter. Le setter remplace la valeur : pour un objet ou un tableau, on produit une **copie** (spread), on ne mute pas.

Quand le nouvel état dépend de l'ancien, on passe une fonction : **setCount((c) => c + 1)**. L'initialisation coûteuse se fait via une fonction **useState(() => loadFavorites())**.

Dans WorldGuide : query de recherche, pays sélectionné, liste des codes favoris.

- Un state par préoccupation (éviter un seul mega-objet trop tôt)
- Mises à jour asynchrones et batchées
- État dérivé : calculer **filtered** pendant le rendu, pas dans un effet
- Remonter l'état (lifting) dès que deux enfants le partagent

```
import { useState } from 'react';

export function useFavorites(initial = []) {
  const [codes, setCodes] = useState(initial);

  function toggle(code) {
    setCodes((current) =>
      current.includes(code)
        ? current.filter((item) => item !== code)
        : [...current, code]
    );
  }

  return { codes, toggle, count: codes.length };
}

export function SearchBox() {
  const [query, setQuery] = useState('');
  return (
    <input
      value={query}
      onChange={(event) => setQuery(event.target.value)}
      placeholder="Pays"
    />
  );
}
```

Effets et cycle de vie avec useEffect

useEffect synchronise React avec un système externe : réseau, DOM, timer, abonnement. Il s'exécute *après* le paint. Le tableau de dépendances contrôle la fréquence.

Tableau vide : montage uniquement. Dépendances : rejouer quand elles changent. Absence de tableau : après chaque rendu (rarement souhaitable). La fonction retournée est le **cleanup**.

Pièges classiques : oublier une dépendance, lancer un fetch sans AbortController, mettre à jour l'état après démontage, ou utiliser un effet pour dériver de l'état (préférer le calcul pendant le rendu).

- Fetch pays + abort au cleanup
- Debounce de la recherche (timer nettoyé)
- Pas d'effet pour filtrer une liste déjà en mémoire
- React 18 Strict Mode : double montage en dev, d'où des effets idempotents

```
import { useEffect, useState } from 'react';

export function useDebouncedValue(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);

  return debounced;
}

export function useCountrySearch(query) {
  const [data, setData] = useState([]);
  const [error, setError] = useState(null);
  const debounced = useDebouncedValue(query);

  useEffect(() => {
    if (!debounced) {
      setData([]);
      return undefined;
    }
    const controller = new AbortController();
    fetch(`https://restcountries.com/v3.1/name/${debounced}?fields=cca2,name`, {
      signal: controller.signal,
    })
      .then((response) => {
        if (!response.ok) throw new Error('Introuvable');
        return response.json();
      })
      .then(setData)
      .catch((err) => {
        if (err.name !== 'AbortError') setError(err.message);
      });
    return () => controller.abort();
  }, [debounced]);

  return { data, error };
}
```

Context API pour l’état global

Le Context transporte une valeur à travers l'arbre sans la faire passer de props en props. Il convient au **thème**, à la **locale**, à l'utilisateur connecté, pas à un état qui change à chaque frappe.

On crée un contexte, un Provider, et un hook **useWorldGuide()** qui lève une erreur hors provider. Découper les contextes (thème vs favoris) limite les re-renders.

Tout consommateur se re-rend quand la value change : mémoïser la value du Provider (**useMemo**) si elle est un objet.

- Bon : thème, auth, i18n
- Moyen : favoris WorldGuide si peu de mises à jour
- Mauvais : query de recherche (trop fréquente)
- Toujours encapsuler dans un hook métier

```
import { createContext, useContext, useMemo, useState } from 'react';

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const value = useMemo(() => ({ theme, setTheme }), [theme]);
  return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>;
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme doit être utilisé dans ThemeProvider');
  return ctx;
}
```

Création de hooks personnalisés

Un hook personnalisé extrait une logique stateful réutilisable. Il commence par **use**, compose d'autres hooks, et retourne un tuple ou un objet clair.

useCountries(region), **useDebouncedValue(query)**, **useLocalStorage('favorites')** rendent les pages WorldGuide minces et testables isolément.

Ne pas cacher trop d'effets de bord : un hook de fetch doit exposer **status**, **error** et **data**.

- Nom préfixé par use, règles des hooks inchangées
- Retourner data / error / isLoading
- Un hook = une histoire utilisateur
- Tester le hook avec renderHook (Testing Library)

```
import { useEffect, useState } from 'react';

export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const raw = window.localStorage.getItem(key);
      return raw ? JSON.parse(raw) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    window.localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

export function useCountries(region) {
  const [state, setState] = useState({ status: 'idle', data: [], error: null });

  useEffect(() => {
    const controller = new AbortController();
    setState({ status: 'loading', data: [], error: null });
    const url = region
      ? `https://restcountries.com/v3.1/region/${region}?fields=cca2,name,capital`
      : 'https://restcountries.com/v3.1/all?fields=cca2,name,capital';
    fetch(url, { signal: controller.signal })
      .then((r) => r.json())
      .then((data) => setState({ status: 'success', data, error: null }))
      .catch((error) => {
        if (error.name !== 'AbortError') {
          setState({ status: 'error', data: [], error: error.message });
        }
      });
    return () => controller.abort();
  }, [region]);

  return state;
}
```

Patterns de gestion d’état

On choisit l'outil selon la **portée** et la **fréquence** des mises à jour : useState local, useReducer pour des transitions nommées, Context pour du global rare, une lib (Redux Toolkit, Zustand, Jotai) pour du partagé complexe, React Query pour le serveur.

L'état serveur (liste des pays) n'est pas du state React classique : cache, stale time, invalidation. L'état UI (modale ouverte, onglet actif) reste local.

WorldGuide : query locale, favoris en localStorage via hook, catalogue via fetch/React Query, auth éventuelle en Context.

- Séparer état serveur et état UI
- useReducer + actions nommées dès que les transitions se complexifient
- Éviter de dupliquer le cache HTTP dans Redux sans raison
- Colocaliser l'état au plus proche de son usage

```
export function uiReducer(state, action) {
  switch (action.type) {
    case 'openDetail':
      return { ...state, selectedCode: action.code, modal: true };
    case 'closeDetail':
      return { ...state, modal: false };
    default:
      return state;
  }
}

export function classifyState({ query, favorites, countries }) {
  return {
    ui: { query },
    client: { favorites },
    server: { countries },
  };
}
```

Communication entre composants

Analyse des patterns de communication

Les composants communiquent principalement par **props** (parent → enfant) et **callbacks** (enfant → parent). Les frères communiquent via un parent commun (état levé).

D'autres canaux existent : Context, stores, URL (source de vérité pour le pays affiché), et plus rarement des events DOM custom.

Règle WorldGuide : le code pays sélectionné vit dans l'URL dès qu'on a un router ; le filtre peut rester local à la page liste.

- Props down / events up : cas par défaut
- Lifting state : frères (filtre + liste)
- URL : partageable, bookmarkable
- Store : quand l'arbre devient trop profond

```
export function ParentList() {
  const countries = [{ code: 'FR', name: 'France' }];
  function handleSelect(code) {
    console.log('pays choisi', code);
  }
  return <CountryButtons countries={countries} onSelect={handleSelect} />;
}

function CountryButtons({ countries, onSelect }) {
  return countries.map((country) => (
    <button key={country.code} type="button" onClick={() => onSelect(country.code)}>
      {country.name}
    </button>
  ));
}

export { CountryButtons };
```

Solutions au props drilling

Le **props drilling** consiste à faire transiter des props à travers des composants qui ne les utilisent pas. Trois niveaux de CountryLayout → Sidebar → Card rendent le refactor pénible.

Solutions : composition (passer déjà le bon enfant), Context, colocation de l'état, ou un store. Composition est souvent la plus simple : le parent crée **<FavoriteButton />** et le glisse en children.

Ne pas introduire Redux uniquement pour éviter trois props.

- Composer plutôt que forer
- Context ciblé (FavoritesProvider)
- Découper l'arbre pour rapprocher état et usage
- Component injection / slots

```
export function LayoutWithDrilling({ favorites, onToggle, children }) {
  return <Shell favorites={favorites} onToggle={onToggle}>{children}</Shell>;
}

function Shell({ children }) {
  return <div className="shell">{children}</div>;
}

export function LayoutComposed({ sidebar, children }) {
  return (
    <div className="shell">
      <aside>{sidebar}</aside>
      <main>{children}</main>
    </div>
  );
}
```

Utilisation avancée du context

Un Context avancé sépare **état** et **dispatch** en deux providers : les boutons qui dispatchent ne se re-rendent pas quand la liste des favoris change.

On peut aussi sélectionner une tranche avec des bibliothèques (use-context-selector) ou accepter Redux/Zustand pour des sélecteurs fins.

WorldGuide : **FavoritesStateContext** + **FavoritesDispatchContext**, hook **useIsFavorite(code)**.

- Split state / dispatch
- Mémoïser la value objet
- Hooks qui throw hors provider
- Ne pas y mettre le résultat brut de fetch 250 pays si chaque frappe re-rend tout

```
import { createContext, useContext, useMemo, useReducer } from 'react';

const StateCtx = createContext(null);
const DispatchCtx = createContext(null);

function reducer(state, action) {
  if (action.type === 'toggle') {
    const has = state.includes(action.code);
    return has ? state.filter((c) => c !== action.code) : [...state, action.code];
  }
  return state;
}

export function FavoritesProvider({ children }) {
  const [state, dispatch] = useReducer(reducer, []);
  const codes = useMemo(() => state, [state]);
  return (
    <DispatchCtx.Provider value={dispatch}>
      <StateCtx.Provider value={codes}>{children}</StateCtx.Provider>
    </DispatchCtx.Provider>
  );
}

export function useFavoritesCodes() {
  return useContext(StateCtx);
}

export function useToggleFavorite() {
  return useContext(DispatchCtx);
}
```

Patterns de composition et render props

Les **render props** consistent à passer une fonction **children(data)** (ou **render**) pour déléguer le rendu. C'était le standard avant les hooks pour partager un fetch ou un suivi souris.

Aujourd'hui, un hook **useCountries()** est plus lisible. Les render props restent utiles pour des bibliothèques de headless UI (Downshift, React Aria) où le rendu doit rester 100 % contrôlé par l'app.

WorldGuide peut exposer **<CountryData render={({ countries }) => ...} />** puis montrer l'équivalent hook.

- Render props : flexibilité maximale du JSX
- Hooks : même logique, moins de nesting
- Headless components : comportement sans markup imposé
- HOC : wrapping historique, signatures parfois obscures

```
export function CountryData({ children }) {
  const countries = [{ code: 'JP', name: 'Japon' }];
  return children({ countries });
}

export function CountryListWithRenderProp() {
  return (
    <CountryData>
      {({ countries }) => (
        <ul>
          {countries.map((c) => (
            <li key={c.code}>{c.name}</li>
          ))}
        </ul>
      )}
    </CountryData>
  );
}
```

Gestion des événements complexes

Les interactions riches (drag d'une carte, raccourcis clavier, formulaires multi-étapes) demandent de coordonner plusieurs handlers et parfois un **useReducer** pour les transitions.

On normalise : un événement UI devient une intention métier (**FAVORITE_TOGGLED**, **SEARCH_SUBMITTED**). Le reducer reste testable sans DOM.

Pour le clavier dans la liste WorldGuide : flèches, Enter pour ouvrir la fiche, Escape pour fermer.

- Intentions nommées plutôt que logique dans le JSX
- stopPropagation seulement avec une raison claire
- Gestion du focus après navigation
- Debounce / throttle sur resize et input

```
export function listHotkeysReducer(state, action) {
  const max = state.items.length - 1;
  switch (action.type) {
    case 'ArrowDown':
      return { ...state, index: Math.min(state.index + 1, max) };
    case 'ArrowUp':
      return { ...state, index: Math.max(state.index - 1, 0) };
    case 'Enter':
      return { ...state, opened: state.items[state.index] };
    case 'Escape':
      return { ...state, opened: null };
    default:
      return state;
  }
}

export function onListKeyDown(dispatch, event) {
  if (['ArrowDown', 'ArrowUp', 'Enter', 'Escape'].includes(event.key)) {
    event.preventDefault();
    dispatch({ type: event.key });
  }
}
```

Gestion avancée de l'état

Architecture Redux et principes fondamentaux

Redux centralise l'état dans un **store** unique, mis à jour de façon **pure** par des reducers. Trois principes : une source de vérité, l'état en lecture seule, des changements décrits par des actions.

Le flux est prévisible : UI dispatch une action → reducer retourne un nouvel état → les sélecteurs dérivent les données → les composants s'abonnent.

On l'introduit dans WorldGuide quand favoris, filtres persistés et panier d'itinéraires doivent être inspectables (DevTools) et partagés hors de l'arbre React (middleware).

- Store unique, reducers composables
- Actions : objets { type, payload }
- Reducers purs : pas de fetch dedans
- Redux DevTools : time-travel

```
export function createStore(reducer, initialState) {
  let state = initialState;
  const listeners = new Set();
  return {
    getState: () => state,
    dispatch: (action) => {
      state = reducer(state, action);
      listeners.forEach((fn) => fn());
    },
    subscribe: (fn) => {
      listeners.add(fn);
      return () => listeners.delete(fn);
    },
  };
}

export const initialWorldGuide = { favorites: [] };
```

Implémentation des actions et reducers

Une action décrit *ce qui s'est passé*, pas comment muter. Le reducer, fonction **(state, action) => newState**, applique le changement de façon immuable.

On factorise avec des action creators. Les types string magiques se transforment, avec Redux Toolkit, en slices. En vanilla, on combine les reducers **favorites**, **filters**, **user**.

Toujours retourner un nouvel objet : spread, pas **state.push**.

- Action creator : **toggleFavorite(code)**
- Reducer exhaustif avec default
- combineReducers pour découper le store
- Pas d'effets de bord dans le reducer

```
export const toggleFavorite = (code) => ({ type: 'favorites/toggle', payload: code });

export function favoritesReducer(state = [], action) {
  switch (action.type) {
    case 'favorites/toggle':
      return state.includes(action.payload)
        ? state.filter((code) => code !== action.payload)
        : [...state, action.payload];
    default:
      return state;
  }
}
```

Configuration et utilisation des middlewares

Un middleware s'intercale entre le dispatch et le reducer : logging, crash reporting, asynchrone. L'API est **store => next => action**.

redux-thunk autorise de dispatcher une fonction (fetch des pays). **redux-saga** / listeners RTK gèrent des flux plus complexes. Le logger n'est activé qu'en développement.

Aujourd'hui, RTK Query ou React Query remplacent souvent les thunks de fetch. Un middleware reste pertinent pour analytics : chaque **toggleFavorite** envoie un événement.

- Thunk : async dispatch
- Logger : debug du flux
- ListenerMiddleware RTK : réactions déclaratives
- Ne pas bloquer le thread trop longtemps

```
export const logger = (store) => (next) => (action) => {
  console.log('dispatch', action.type, action);
  const result = next(action);
  console.log('state', store.getState());
  return result;
};

export const thunk = (store) => (next) => (action) => {
  if (typeof action === 'function') {
    return action(store.dispatch, store.getState);
  }
  return next(action);
};

export const fetchCountries = () => async (dispatch) => {
  const data = await fetch('https://restcountries.com/v3.1/all?fields=cca2,name').then((r) =>
r.json());
  dispatch({ type: 'countries/loaded', payload: data });
};
```

Optimisation avec Redux Toolkit

Redux Toolkit (RTK) est le standard officiel : **configureStore**, **createSlice** (Immer pour « muter » en toute sécurité), DevTools déjà branchés, thunk inclus.

Un slice WorldGuide **favoritesSlice** déclare **toggle**, **clear** et l'état initial. Moins de boilerplate, moins d'erreurs d'immuabilité.

RTK Query peut cacher le catalogue REST Countries ; on le verra aussi avec TanStack Query, souvent plus idiomatique côté React seul.

- createSlice + Immer
- configureStore : middlewares par défaut
- createAsyncThunk si on reste 100 % Redux pour l'API
- Moins de fichiers actions/types/reducers séparés

```
import { configureStore, createSlice } from '@reduxjs/toolkit';

const favoritesSlice = createSlice({
  name: 'favorites',
  initialState: { codes: [] },
  reducers: {
    toggle(state, action) {
      const code = action.payload;
      const i = state.codes.indexOf(code);
      if (i >= 0) state.codes.splice(i, 1);
      else state.codes.push(code);
    },
    clear(state) {
      state.codes = [];
    },
  },
});

export const { toggle, clear } = favoritesSlice.actions;

export const store = configureStore({
  reducer: { favorites: favoritesSlice.reducer },
});
```

Sélecteurs et performances

Un composant qui fait **useSelector(s => s)** se re-rend à chaque action. Les **sélecteurs** extraient une tranche ; **reselect** (intégré à RTK) mémoïse les dérivations coûteuses (pays filtrés + favoris).

Égalité de référence : retourner un tableau filtré inline crée un nouvel array à chaque fois. Mieux : sélecteur mémoïsé ou comparaison shallow.

WorldGuide : **selectFilteredCountries** dépend de **query** et **countries** uniquement.

- useSelector ciblé
- createSelector pour les listes dérivées
- Normaliser les entités (id → pays) si besoin
- Éviter de stocker du dérivable

```
import { createSelector } from '@reduxjs/toolkit';

const selectCountries = (s) => s.countries.items;
const selectQuery = (s) => s.filters.query;

export const selectFilteredCountries = createSelector(
  [selectCountries, selectQuery],
  (items, query) => {
    const q = query.toLowerCase();
    return items.filter((c) => c.name.common.toLowerCase().includes(q));
  }
);
```

Routing et navigation

Concepts fondamentaux de React Router

React Router synchronise l'**URL** et l'UI. Une SPA WorldGuide passe de **/** (liste) à **/pays/FR** (fiche) sans recharger la page, tout en restant partageable.

Depuis la v6.4, le modèle **data APIs** (createBrowserRouter, loaders, actions) rapproche le router d'un mini-framework. La v6 classique (**BrowserRouter** + **Routes**) reste valable pédagogiquement.

Le routeur ne remplace pas l'état local : il porte ce qui mérite un lien.

- URL = source de vérité pour la ressource affichée
- Nested routes : layout + outlet
- Link / NavLink vs navigate impératif
- History API du navigateur

```
import { BrowserRouter, Link, Route, Routes } from 'react-router-dom';

export function WorldGuideRouterIntro() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Liste</Link>
        <Link to="/favoris">Favoris</Link>
      </nav>
      <Routes>
        <Route path="/" element={<p>Liste des pays</p>} />
        <Route path="/favoris" element={<p>Favoris</p>} />
      </Routes>
    </BrowserRouter>
  );
}
```

Architecture des routes et navigation

On déclare un arbre : layout racine (header WorldGuide), enfants index (liste), **pays/:code** (détail), **favoris**, et une route catch-all 404.

Outlet rend l'enfant dans le layout. **NavLink** ajoute une classe active. La navigation programmatique (**useNavigate**) suit un submit ou un clic carte.

Colocaliser les fichiers de pages aide. Avec les data APIs, chaque route peut avoir son **loader**.

- Layout routes + Outlet
- Index route pour la liste
- NavLink pour l'accessibilité de la nav
- 404 explicite

```
import { NavLink, Outlet } from 'react-router-dom';

export function AppLayout() {
  return (
    <div>
      <header>
        <NavLink to="/" end>WorldGuide</NavLink>
        <NavLink to="/favoris">Favoris</NavLink>
      </header>
      <Outlet />
    </div>
  );
}

export const routeTree = [
  { path: '/', element: <AppLayout />, children: [
    { index: true, element: <p>Liste</p> },
    { path: 'pays/:code', element: <p>Fiche</p> },
    { path: 'favoris', element: <p>Favoris</p> },
    { path: '*', element: <p>Page introuvable</p> },
  ]},
];
```

Gestion avancée des paramètres

useParams lit **:code**. **useSearchParams** gère **?region=europe&q=fr** sans polluer le state React. Les splat ***** capturent le reste du chemin.

Valider le paramètre : si **code** n'est pas un ISO2, rediriger vers 404. Encoder les caractères dans les noms (Côte d'Ivoire) via le code, pas le libellé.

Loaders : **params.code** sert à fetch la fiche avant le rendu (data APIs).

- Params de chemin : ressource
- Search params : filtres partageables
- Validation et redirections
- Types : parser/valider (zod) en bordure

```
import { Navigate, useParams, useSearchParams } from 'react-router-dom';

export function CountryRoute() {
  const { code } = useParams();
  const [params, setParams] = useSearchParams();
  const region = params.get('region');

  if (!code || !/^[A-Z]{2}$/i.test(code)) {
    return <Navigate to="/404" replace />;
  }

  return (
    <section>
      <h1>Pays {code.toUpperCase()}</h1>
      <p>Région filtre : {region || 'toutes'}</p>
      <button type="button" onClick={() => setParams({ region: 'europe' })}>
        Europe
      </button>
    </section>
  );
}
```

Protection et authentification

Une route protégée vérifie une session avant de rendre l'enfant. Pattern : composant **RequireAuth** qui redirige vers **/login** en conservant **location** pour le retour.

Côté data APIs, le loader lance un **redirect('/login')**. Ne jamais cacher un secret dans le front : le vrai contrôle est serveur. Le front évite surtout d'afficher l'UI admin par erreur.

WorldGuide : page **/favoris-cloud** fictive réservée aux utilisateurs connectés.

- Redirect + state.from
- Rôles : RequireRole('editor')
- Loaders pour le garde data-aware
- Tokens : httpOnly cookies de préférence

```
import { Navigate, useLocation } from 'react-router-dom';

export function RequireAuth({ isLoggedIn, children }) {
  const location = useLocation();
  if (!isLoggedIn) {
    return <Navigate to="/login" replace state={{ from: location }} />;
  }
  return children;
}
```

Optimisation du routing

Chaque page peut être un chunk séparé via **React.lazy** + **lazy** de React Router. Les loaders évitent les waterfalls (fetch dans useEffect après paint).

defer / streaming permet d'afficher le layout tout de suite et le détail ensuite. **useNavigation** affiche un indicateur global.

Précharger au hover d'un Link (selon version) améliore le ressenti du guide de pays.

- Lazy routes par page
- Loaders vs useEffect
- Pending UI avec useNavigation
- Éviter les layouts qui remountent la liste

```
import { Suspense, lazy } from 'react';

const CountryMap = lazy(() => import('./CountryMap.jsx'));

export function CountryDetail({ code }) {
  return (
    <Suspense fallback={<p>Chargement de la carte…</p>}>
      <CountryMap code={code} />
    </Suspense>
  );
}

export function CountryMap({ code }) {
  return <div>Carte {code}</div>;
}
```

Performance et optimisation

Mémoisation avec React.memo et useMemo

React.memo évite de re-rendre un composant si ses props n'ont pas changé (comparaison shallow). **useMemo** conserve le résultat d'un calcul entre rendus.

Ce n'est pas gratuit : comparer des props et stocker des caches a un coût. On mémoïse une **CountryCard** dans une liste de 250 éléments, ou un filtre lourd, pas un simple titre.

Si une prop fonction change d'identité à chaque rendu, memo ne sert à rien : d'où **useCallback** au chapitre suivant.

- Profiler d'abord, mémoïser ensuite
- memo sur les items de liste
- useMemo pour filtrages / tris coûteux
- Objets inline ({ theme }) cassent memo

```
import { memo, useMemo } from 'react';

export const CountryCard = memo(function CountryCard({ country, onSelect }) {
  return (
    <button type="button" onClick={() => onSelect(country.code)}>
      {country.name.common}
    </button>
  );
});

export function useFiltered(countries, query) {
  return useMemo(() => {
    const q = query.toLowerCase();
    return countries.filter((c) => c.name.common.toLowerCase().includes(q));
  }, [countries, query]);
}
```

Optimisation des callbacks avec useCallback

useCallback mémorise une fonction. Utile uniquement si l'enfant est mémoïsé, ou si la fonction est dépendance d'un effet.

Préférer **setState(fn)** pour des setters stables. Un callback par carte (**() => onSelect(code)**) recrée N fonctions : parfois on passe **code** en data-attribute, ou on mémoïse avec le code en dépendance.

Ne pas wrapper tous les handlers « par habitude ».

- Coupler memo + useCallback
- Dépendances exactes, comme useEffect
- Setters de useState déjà stables
- Éviter la sur-optimisation

```
import { useCallback, useState } from 'react';
import { CountryCard } from './react_memoization.js';

export function CountryGrid({ countries }) {
  const [selected, setSelected] = useState(null);
  const onSelect = useCallback((code) => setSelected(code), []);

  return (
    <div>
      {countries.map((country) => (
        <CountryCard key={country.cca2} country={country} onSelect={onSelect} />
      ))}
      <p>Sélection : {selected}</p>
    </div>
  );
}
```

Techniques de code splitting

Le code splitting découpe le bundle pour ne télécharger que ce qui est nécessaire. Vite le fait via **import()** dynamique.

On split par route (page détail, page admin) et par librairie lourde (carte Leaflet, graphique). Le vendor React reste dans un chunk commun.

Analyser le bundle (**rollup-plugin-visualizer**) avant d'optimiser à l'aveugle.

- Split par route
- import() des libs optionnelles
- Éviter les imports circulaires qui forcent un gros chunk
- Mesurer : Lighthouse, preview production

```
export async function loadLeaflet() {
  const mod = await import('leaflet');
  return mod;
}

export const splitHint = `Dans Vite, import() crée un chunk automatique.
Lazy-loader les pages /pays et la lib carte, pas le header.`;
```

Implémentation du lazy loading

React.lazy charge un composant au premier rendu, enveloppé dans **Suspense** avec un fallback. React Router **lazy** fait de même au niveau route.

Lazy-load des images de drapeaux (loading=lazy natif) et des cartes. Attention au CLS : réserver la hauteur.

WorldGuide : **CountryMap** chargé seulement sur la fiche détail.

- lazy + Suspense
- Error boundary autour des chunks (échec réseau)
- Préchargement intelligent
- Pas de lazy sur des composants au-dessus de la ligne de flottaison critiques

```
import { Suspense, lazy } from 'react';

const HeavyChart = lazy(() => import('./HeavyChart.jsx'));

export function PopulationPanel({ show, data }) {
  if (!show) return null;
  return (
    <Suspense fallback={<p>Graphique…</p>}>
      <HeavyChart data={data} />
    </Suspense>
  );
}

export function HeavyChart({ data }) {
  return <pre>{JSON.stringify(data.slice(0, 3), null, 2)}</pre>;
}
```

Outils de profiling et debugging

Le **Profiler** React mesure la durée des rendus. On cherche les commits longs après une frappe dans la recherche.

Chrome Performance complète pour le main thread. **why-did-you-render** (dev) signale les re-renders évitables.

Objectif : interaction < 100 ms ressentie sur le filtre WorldGuide.

- Profiler : flamegraph, ranked
- Highlight updates (DevTools)
- Production build pour mesurer vraiment
- Web Vitals : LCP, INP, CLS

```
import { Profiler } from 'react';

export function onRender(id, phase, actualDuration) {
  if (actualDuration > 16) {
    console.warn(`${id} ${phase} ${actualDuration.toFixed(1)}ms`);
  }
}

export function ProfiledList({ children }) {
  return (
    <Profiler id="CountryList" onRender={onRender}>
      {children}
    </Profiler>
  );
}
```

Test et débogage

Tests avec Jest et React Testing Library

React Testing Library pousse à tester comme un utilisateur : roles, labels, texte visible — pas les détails d'implémentation (state interne).

Jest ou Vitest exécute les tests. On rend **CountryCard**, on clique « Ajouter aux favoris », on assert le texte « ★ favori ».

userEvent simule mieux que fireEvent (tab, frappe réelle).

- Queries : getByRole, getByLabelText
- userEvent.click / type
- await findBy* pour l'async
- Éviter snapshot massifs de tout le DOM

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { CountryCard } from './react_props_validation.js';

test('ajoute un pays aux favoris', async () => {
  const user = userEvent.setup();
  const onToggle = vi.fn();
  render(
    <CountryCard
      country={{ code: 'FR', name: 'France' }}
      onToggleFavorite={onToggle}
    />
  );
  await user.click(screen.getByRole('button', { name: 'Ajouter' }));
  expect(onToggle).toHaveBeenCalledWith('FR');
});
```

Stratégies de test unitaire et d’intégration

La pyramide : beaucoup de tests unitaires (hooks, reducers), des tests d'intégration (page liste + mock HTTP), peu d'E2E (Playwright) sur le parcours critique.

Tester le comportement : filtrer la France, ouvrir la fiche, ajouter un favori. Pas la structure CSS.

Les tests E2E WorldGuide couvrent le happy path du guide de pays.

- Unitaire : reducer, formatage population
- Intégration : page + router + mock API
- E2E : 1 ou 2 parcours métier
- Contrats API (MSW) partagés

```
export const testLayers = {
  unit: ['favoritesReducer', 'selectFilteredCountries', 'formatPopulation'],
  integration: ['CountryListPage + MSW', 'router /pays/:code'],
  e2e: ['ouvrir le Japon depuis la liste'],
};

export function formatPopulation(n) {
  return new Intl.NumberFormat('fr-FR').format(n);
}
```

Mocking et simulation de données

Mock Service Worker (MSW) intercepte le réseau en tests et en dev. Les composants utilisent le vrai **fetch** ; on ne mocke pas Axios en profondeur.

Fixtures : un petit jeu de pays (FR, JP, BR). Simuler 404, 500, latence pour les états d'erreur et de chargement.

vi.mock / jest.mock pour les modules non déterministes (dates, analytics).

- MSW handlers REST Countries
- Fixtures minimales réalistes
- Pas de mock du Router sauf besoin
- Horloge fake pour debounce

```
import { http, HttpResponse } from 'msw';

export const handlers = [
  http.get('https://restcountries.com/v3.1/alpha/:code', ({ params }) => {
    if (params.code === 'XX') {
      return HttpResponse.json({ message: 'Not Found' }, { status: 404 });
    }
    return HttpResponse.json({
      cca2: 'FR',
      name: { common: 'France' },
      capital: ['Paris'],
    });
  }),
];
```

Techniques de débogage avancées

Au-delà de console.log : breakpoints conditionnels, logpoints, traces de renders, replay (quand disponible), et lecture des source maps.

Les bugs React fréquents : stale closure dans un effet, key instable, setState après unmount, état dérivé mal copié.

Reproduire WorldGuide avec React DevTools + un breakpoint dans le reducer favoris.

- Stale closures : dépendances
- Compare prev / next props dans memo
- Network tab : CORS, HTTP
- Error overlay Vite

```
export function explainStaleClosure() {
  return [
    'Un effet capture query au moment de sa création.',
    'Si query n\'est pas dans le tableau de dépendances, le fetch utilise une valeur périmée.',
    'Correctif : ajouter query aux deps, ou lire une ref à jour.',
  ].join(' ');
}

export function isLikelyKeyBug(prevList, nextList) {
  return prevList.some((item, i) => item.code !== nextList[i]?.code);
}
```

Gestion des erreurs avec Error Boundaries

Un **Error Boundary** est un composant classe (ou lib équivalente) qui attrape les erreurs de rendu des enfants et affiche un fallback. Il ne capture pas les erreurs dans les handlers d'événements ni les async (il faut un try/catch ou un error state).

On place une boundary autour de la fiche pays (échec d'un widget carte) pour ne pas faire tomber toute l'app.

Coupler avec un reporting (Sentry) dans **componentDidCatch**.

- getDerivedStateFromError + fallback UI
- Périmètre : widget, pas toute la root seulement
- Erreurs async : état error dédié
- Bouton réessayer qui reset la boundary

```
import { Component } from 'react';

export class MapErrorBoundary extends Component {
  state = { error: null };

  static getDerivedStateFromError(error) {
    return { error };
  }

  componentDidCatch(error, info) {
    console.error('map-widget', error, info.componentStack);
  }

  render() {
    if (this.state.error) {
      return (
        <div role="alert">
          <p>Impossible d'afficher la carte.</p>
          <button type="button" onClick={() => this.setState({ error: null })}>
            Réessayer
          </button>
        </div>
      );
    }
    return this.props.children;
  }
}
```

React Server Components

Architecture des Server Components

Les **React Server Components (RSC)** s'exécutent sur le serveur (ou à build time) et envoient un format sérialisé au client, pas du HTML figé seulement : l'arbre peut contenir des Client Components aux feuilles interactives.

Ils peuvent lire une base, un filesystem, des secrets, sans les embarquer dans le bundle. Pas de useState ni d'effets dans un Server Component.

WorldGuide en Next.js : la liste des pays est un Server Component qui await l'API ; le bouton favori est un Client Component.

- Zéro JS client pour les parties statiques
- Accès data direct au serveur
- Composition SC + CC
- Nécessite un framework compatible (Next.js App Router)

```
// Illustration RSC : composant async côté serveur (Next.js)
export default async function CountryIndex() {
  const countries = await fetch(
    'https://restcountries.com/v3.1/all?fields=cca2,name',
    { next: { revalidate: 3600 } }
  ).then((r) => r.json());

  return (
    <ul>
      {countries.map((c) => (
        <li key={c.cca2}>{c.name.common}</li>
      ))}
    </ul>
  );
}
```

Différences entre Server et Client Components

Par défaut, dans l'App Router Next.js, un fichier est un **Server Component**. On ajoute **'use client'** dès qu'on a besoin d'état, d'effets, d'événements navigateur.

Un Server Component peut importer un Client Component, pas l'inverse pour du code serveur (secrets). On passe des props sérialisables (pas de fonctions, sauf celles déjà côté client via composition).

Règle : laisser le maximum en serveur ; marquer client le plus bas possible (bouton, champ recherche).

- SC : async, I/O, pas d'état UI
- CC : hooks, onClick, APIs browser
- use client à la frontière
- Props sérialisables à travers la frontière

```
'use client';

import { useState } from 'react';

export function FavoriteButton({ code }) {
  const [on, setOn] = useState(false);
  return (
    <button type="button" onClick={() => setOn((v) => !v)}>
      {on ? '★' : '☆'} {code}
    </button>
  );
}

export function ServerPageHint() {
  return 'Pas de useState ici : ce fichier reste un Server Component et importe
FavoriteButton.';
}
```

Implémentation du Streaming SSR

Le **streaming** envoie le HTML par morceaux : le shell (nav) arrive d'abord, les Suspense boundaries se remplissent ensuite. L'utilisateur voit WorldGuide plus tôt.

SSR classique attendait toute la data. Avec RSC + Suspense, une fiche lente (météo) n'bloque pas le titre du pays.

Attention au buffering des proxies (CDN) qui peut casser le streaming si mal configuré.

- Suspense comme unité de stream
- Shell immédiat, data progressive
- loading.js dans Next.js
- Mesurer TTFB et LCP

```
import { Suspense } from 'react';

async function Weather({ capital }) {
  const weather = await fetch(`https://example.invalid/weather/${capital}`).catch(() => null);
  return <p>Météo {capital} : {weather ? 'ok' : 'indisponible'}</p>;
}

export function CountryShell({ name, capital }) {
  return (
    <>
      <h1>{name}</h1>
      <Suspense fallback={<p>Chargement météo…</p>}>
        <Weather capital={capital} />
      </Suspense>
    </>
  );
}
```

Stratégies de data fetching

En RSC, on fetch directement dans le composant async, éventuellement avec un cache (Next **fetch** étendu). On évite l'eau en cascade : parallèle via **Promise.all**, ou composition de composants chacun suspendu.

Côté client, TanStack Query reste pertinent pour les données très interactives (recherche instantanée).

WorldGuide : liste cachée 1 h (revalidate), détail à la demande, recherche client.

- Fetch dans le Server Component
- Éviter waterfalls parent/enfant
- Cache et revalidate
- Client fetch pour l'instantané

```
export async function loadCountryBundle(code) {
  const [country, neighbors] = await Promise.all([
    fetch(`https://restcountries.com/v3.1/alpha/${code}`).then((r) => r.json()),
    fetch(`https://restcountries.com/v3.1/alpha/${code}?fields=borders`).then((r) => r.json()),
  ]);
  return { country: country[0], neighbors };
}
```

Performance et hydration

L'**hydration** attache les listeners au HTML déjà envoyé. Moins il y a de JS client, plus elle est rapide. Les RSC réduisent la surface à hydrater.

Une mismatch (HTML serveur ≠ premier rendu client) provoque un warning. Causes : Date.now(), Math.random(), window dans le rendu.

Selective hydration (React 18) priorise les zones avec lesquelles l'utilisateur interagit.

- Réduire 'use client'
- Pas d'accès window pendant le SSR
- suppressHydrationWarning avec parcimonie
- Islands : hydrater le search, pas toute la liste statique

```
export function ClientClockWrong() {
  return <time>{new Date().toLocaleString()}</time>;
}

export function ClientClockRight({ iso }) {
  return <time dateTime={iso}>{iso}</time>;
}

export const hydrationTips = [
  'Ne pas lire window pendant le rendu serveur',
  'Passer la date depuis le serveur en prop',
  'Réduire les Client Components pour hydrater moins',
];
```

Intégration avec Next.js

L'**App Router** (dossier **app/**) est le véhicule officiel des RSC : layouts imbriqués, loading, error, route handlers.

WorldGuide en Next : **app/page.js** liste serveur, **app/pays/[code]/page.js** fiche, **app/pays/[code]/error.js** fallback, client components dans **components/**.

On n'a pas besoin de Next pour tout le cours : Vite reste le bon outil SPA. Next entre en jeu pour SEO, streaming et data serveur.

- app/ vs pages/
- Layouts persistants
- Route Handlers pour BFF
- Metadata API pour le SEO des fiches pays

```
export const nextAppTree = {
  'app/layout.js': 'header WorldGuide + children',
  'app/page.js': 'liste (Server Component)',
  'app/pays/[code]/page.js': 'fiche',
  'app/pays/[code]/error.js': 'fallback erreur',
  'app/pays/[code]/loading.js': 'skeleton',
};

export function countryMetadata(country) {
  return {
    title: `${country.name.common} — WorldGuide`,
    description: `Capitale : ${country.capital?.[0] ?? 'n/a'}`,
  };
}
```

Intégration API et gestion des données

Choix et configuration des clients HTTP

fetch natif suffit souvent. Axios apporte interceptors et timeout historiques. ky, ofetch sont des wrappers modernes. L'important : une **instance unique** (baseURL, headers, gestion 401).

Annulation via AbortController. Typage des réponses. Ne pas dupliquer la logique d'erreur dans chaque page.

WorldGuide : petit module **api.js** autour de REST Countries.

- fetch + wrapper métier
- Timeouts et abort
- Intercepteurs auth
- Erreurs normalisées

```
const BASE = import.meta.env?.VITE_COUNTRIES_API ??
'https://restcountries.com/v3.1';

export async function api(path, { signal } = {}) {
  const response = await fetch(`${BASE}${path}`, { signal });
  if (!response.ok) {
    const error = new Error(`HTTP ${response.status}`);
    error.status = response.status;
    throw error;
  }
  return response.json();
}

export const countriesApi = {
  all: (fields = 'cca2,name,capital,population,flags') =>
    api(`/all?fields=${fields}`),
  byCode: (code) => api(`/alpha/${code}`),
  byName: (q) => api(`/name/${encodeURIComponent(q)}?fields=cca2,name`),
};
```

Architecture React Query

TanStack Query (React Query) gère le cache serveur : clés de requête, états isLoading / isFetching / isError, retries, déduplication.

Le QueryClientProvider enveloppe l'app. **useQuery({ queryKey: ['countries', region], queryFn })** remplace un useEffect fetch artisanal.

Mutations (**useMutation**) pour plus tard (notes personnelles sur un pays).

- queryKey = identité du cache
- staleTime vs cacheTime/gcTime
- Devtools TanStack
- Pas un remplacement de Redux pour l'état UI

```
import { QueryClient, useQuery } from '@tanstack/react-query';
import { countriesApi } from './react_api_clients.js';

export const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 60_000, retry: 1 } },
});

export function useCountries() {
  return useQuery({
    queryKey: ['countries'],
    queryFn: () => countriesApi.all(),
  });
}
```

Stratégies de cache et invalidation

Une donnée est **fresh** puis **stale** mais encore affichée. **invalidateQueries** marque stale et refetch. **setQueryData** met à jour le cache sans aller au réseau.

WorldGuide : liste **['countries']** invalidée après rarement ; détail **['country', code]** indépendant.

prefetchQuery au hover d'une carte accélère la navigation.

- Hiérarchie des keys
- invalidate vs setQueryData
- prefetch
- gcTime pour libérer la mémoire

```
export async function prefetchCountry(queryClient, code) {
  await queryClient.prefetchQuery({
    queryKey: ['country', code],
    queryFn: () => fetch(`https://restcountries.com/v3.1/alpha/${code}`).then((r) => r.json()),
  });
}

export function optimisticToggle(queryClient, code) {
  queryClient.setQueryData(['favorites'], (prev = []) =>
    prev.includes(code) ? prev.filter((c) => c !== code) : [...prev, code]
  );
}
```

Gestion avancée des erreurs

On distingue erreurs réseau, 4xx métier (pays introuvable) et 5xx. L'UI propose retry, message clair, et éventuellement un fallback cache.

React Query : **throwOnError**, error boundaries, **meta** pour du reporting. Ne pas toaster 10 fois le même 500.

Cartographier les codes REST Countries (404 name not found) vers un empty state « Aucun pays ».

- Taxonomie des erreurs
- Retry exponentiel limité
- Empty vs error vs offline
- Centraliser le mapping API → UI

```
export function toUiError(error) {
  if (error.status === 404) return { kind: 'empty', message: 'Aucun pays trouvé' };
  if (error.status >= 500) return { kind: 'server', message: 'Service indisponible, réessayez'
};
  if (error.name === 'AbortError') return { kind: 'abort', message: null };
  return { kind: 'network', message: 'Vérifiez votre connexion' };
}
```

Patterns d’optimistic updates

Une mise à jour **optimiste** applique le résultat attendu avant la réponse serveur, puis rollback en cas d'échec. Idéal pour un toggle favori ressenti comme instantané.

TanStack Query : **onMutate** snapshot, **onError** restore, **onSettled** invalidate.

Ne pas l'utiliser si le serveur est la seule source de vérité complexe (prix, stock).

- Snapshot du cache
- Rollback
- Idempotence de la mutation
- Feedback d'échec visible

```
export async function toggleFavoriteMutation(queryClient, { code, add }) {
  await queryClient.cancelQueries({ queryKey: ['favorites'] });
  const previous = queryClient.getQueryData(['favorites']);
  queryClient.setQueryData(['favorites'], (prev = []) =>
    add ? [...new Set([...prev, code])] : prev.filter((c) => c !== code)
  );
  try {
    await fetch('/api/favorites', { method: 'POST', body: JSON.stringify({ code, add }) });
  } catch (error) {
    queryClient.setQueryData(['favorites'], previous);
    throw error;
  }
}
```

Déploiement et CI/CD

Stratégies de build et optimisation

vite build produit des assets hashés, minifiés, tree-shakés. On vérifie la taille du bundle, le split, et le mode preview identique à la prod.

Sourcemaps : hidden en prod publique. Compression gzip/brotli côté CDN. Images : formats modernes, tailles adaptées aux drapeaux.

WorldGuide : budget JS initial raisonnable (< 200 ko gzip hors cartes lazy).

- Analyse du bundle
- Tree-shaking et sideEffects
- Hash de cache
- Preview avant release

```
export const buildChecklist = [
  'npm run lint',
  'npm test -- --run',
  'npm run build',
  'npm run preview',
  'Contrôler la taille dist/assets/*.js',
];

export function assetName(hash, file) {
  return `${file}.${hash}.js`;
}
```

Configuration multi-environnements

Vite expose **import.meta.env.VITE_***. On sépare **.env**, **.env.staging**, **.env.production**. Jamais de secret dans une variable **VITE_** (elle part au client).

API URL, feature flags, clés publiques analytics. Le backend garde les secrets.

WorldGuide : **VITE_COUNTRIES_API** pour pointer vers un mock local en TP.

- Préfixe VITE_ uniquement pour le public
- Fichiers env non commités s'ils contiennent des secrets
- Types pour ImportMetaEnv
- Même contrat d'API entre staging et prod

```
export function getApiBase() {
  return import.meta.env.VITE_COUNTRIES_API || 'https://restcountries.com/v3.1';
}

export const envExample = `
VITE_COUNTRIES_API=https://restcountries.com/v3.1
VITE_ENABLE_MAP=false
`;
```

Mise en place du CI/CD

Un pipeline type : install (lockfile), lint, test, build, puis déploiement (Netlify, Vercel, S3+CloudFront, Nginx). Les PR ont un preview.

GitHub Actions : cache npm, node version pinée. Bloquer le merge si les tests échouent.

WorldGuide : job **verify** puis **deploy** sur main.

- CI verte obligatoire
- Preview deployments
- Cache des dépendances
- Artefact du build

```
export const githubActions = `
name: verify
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm test -- --run
      - run: npm run build
`;
```

Monitoring et analytics

On observe les **Web Vitals**, les erreurs runtime (Sentry) et quelques événements métier (fiche ouverte, favori ajouté) sans tracker personnellement sans consentement.

Source maps uploadées pour des stacktraces lisibles. Sampling pour ne pas exploser la facture.

Un dashboard WorldGuide : taux d'erreur fetch, LCP de la liste.

- RUM / Web Vitals
- Error tracking
- Analytics consentis (RGPD)
- Logs serveur / BFF

```
export function reportWebVital(metric) {
  if (metric.name === 'INP' && metric.value > 200) {
    console.warn('INP élevé', metric.value);
  }
}

export function trackFavorite(code) {
  window.dispatchEvent(new CustomEvent('wg:favorite', { detail: { code } }));
}
```

Sécurité et bonnes pratiques

Le front n'est jamais une barrière de sécurité. On se protège quand même : pas de XSS (**dangerouslySetInnerHTML**), pas de secrets, headers CSP, dépendances scannées, cookies httpOnly pour les sessions.

Valider les params d'URL. Sanitizer si on affiche du HTML externe (descriptions Wikipedia). CORS est une règle navigateur, pas une auth.

WorldGuide consomme une API publique : rester attentif aux évolutions et au rate limit.

- XSS / injection
- CSP et HTTPS
- Dépendances (npm audit)
- Pas de token dans localStorage si on peut éviter

```
export function safeCountryHtml(description) {
  return description.replace(/</g, '&lt;').replace(/>/g, '&gt;');
}

export const securityChecklist = [
  'Pas de secret dans import.meta.env.VITE_*',
  'Pas de dangerouslySetInnerHTML sur du contenu API',
  'Valider :code (ISO2)',
  'npm audit / Dependabot',
];
```

Projet final et conclusion

Synthèse des concepts React avancés

WorldGuide a empilé : composants, hooks, context, routing, cache serveur, tests et déploiement. Le fil conducteur reste **UI = f(état)**, avec l'URL et le cache comme états de premier plan.

Savoir choisir : state local vs context vs Redux vs Query vs RSC. La bonne réponse est celle qui minimise la complexité pour le besoin actuel.

Revoir le cycle de vie des données : fetch, cache, invalidation, optimistic UI.

- Hooks et composition
- État au bon niveau
- URL comme API utilisateur
- Mesure avant optimisation

```
export const worldGuideStack = {
  ui: 'composants + hooks',
  navigation: 'React Router (/pays/:code)',
  serverState: 'TanStack Query ou RSC',
  clientState: 'favoris (localStorage / slice RTK)',
  quality: 'RTL + MSW + CI',
};

export function nextStepFor(topic) {
  const map = {
    state: 'Séparer UI et cache serveur',
    perf: 'Profiler avant memo',
    data: 'queryKey stables',
  };
  return map[topic] ?? 'Relire react.dev';
}
```

Patterns et best practices

Colocation, noms métier, petits composants, hooks testables, pas de logique dans le JSX, accessibilité dès le premier bouton.

Conventions d'équipe > originalité. Un linter (react-hooks/exhaustive-deps) évite des heures de stale closures.

Documenter les décisions (ADR courtes) : pourquoi Query plutôt que Redux pour les pays.

- Accessibilité (roles, focus)
- Immutabilité
- Frontières d'erreurs
- Revues de PR ciblées

```
export const practices = [
  'Tester le comportement utilisateur',
  'Keys stables',
  'Cleanup des effets',
  'Composition plutôt qu\'héritage',
  'Accessibilité : bouton, label, focus',
];

export function assertNever(x) {
  throw new Error(`Cas non géré: ${x}`);
}
```

Architecture et scalabilité

Scaler une app React, c'est scaler l'**organisation du code** : features folders (**countries/**, **favorites/**), API claire, tests par feature, pas un dossier **components/** fourre-tout de 200 fichiers.

Découper les bundles, limiter l'état global, introduire un BFF si trop de mashup d'APIs dans le navigateur.

WorldGuide peut grandir vers itinéraires, comptes, offline (service worker) sans tout réécrire si les frontières sont nettes.

- Feature-sliced ou équivalent
- Contrats d'API stables
- Perf budgets
- Équipes : ownership par domaine

```
export const featureFolders = [
  'src/features/countries/',
  'src/features/favorites/',
  'src/app/routes.jsx',
  'src/shared/api.js',
];

export function canStayLocal(stateName) {
  return ['modalOpen', 'query', 'highlightedIndex'].includes(stateName);
}
```

Ressources et communauté

Sources de vérité : documentation React, Next.js, TanStack Query, React Router. Le blog react.dev et les RFC GitHub expliquent le « pourquoi ».

Éviter les tutos obsolètes (classes partout, Redux 2016). Vérifier la date et la version.

Communauté : Discord, Stack Overflow, conférences (React Conf, React Paris).

- react.dev
- Overreacted / blogueurs soigneusement choisis
- Changelog des libs majeures
- CanIUse / MDN pour le web sous-jacent

```
export const resources = [
  { name: 'React', url: 'https://react.dev' },
  { name: 'React Router', url: 'https://reactrouter.com' },
  { name: 'TanStack Query', url: 'https://tanstack.com/query' },
  { name: 'Next.js App Router', url: 'https://nextjs.org/docs/app' },
  { name: 'Testing Library', url: 'https://testing-library.com/docs/react-testing-library/intro' },
];
```

Évolutions et perspectives

React évolue vers le serveur (RSC), les transitions, les Actions, le compilateur (React Compiler) qui mémoïse automatiquement, et un meilleur support des assets.

Les frameworks (Next, Remix) absorbent une partie de l'infrastructure. Savoir le React « nu » reste indispensable pour déboguer.

WorldGuide pourra migrer progressivement : SPA Vite aujourd'hui, routes serveur demain, sans jeter les composants métier.

- React Compiler
- Actions / formulaires
- RSC plus répandus
- Accessibilité et perf toujours non négociables

```
export const watchList = [
  'React Compiler (mémoisation automatique)',
  'Actions et formulaires',
  'RSC hors Next (Vite / autres runtimes)',
  'INP comme vital principal',
];

export function migrationPath() {
  return ['SPA Vite', 'ajouter Router + Query', 'isoler les CC', 'option Next app/'];
}
```

React, gérer la navigation de son application web avec react-router

Introduction à React Router

Cette section applique le routing sur un cas concret : un **guide de pays**. On relie tout ce qui précède (liste, fiche, favoris) à des URLs stables.

Sans router, WorldGuide ne peut pas être partagé (« regarde la fiche Japon »). Avec router, chaque pays a une adresse **/pays/JP**.

On utilise React Router v6 : Routes, Route, Link, hooks.

- Pourquoi une SPA a besoin d'URLs
- Installation de react-router-dom
- BrowserRouter à la racine
- Objectif du TP guide de pays

```
export const tpObjectif = {
  app: 'WorldGuide',
  lib: 'react-router-dom v6',
  api: 'https://restcountries.com',
  routes: ['/', '/pays/:code', '/favoris'],
};

export function installHint() {
  return 'npm install react-router-dom';
}
```

Prise en main de React Router

On enveloppe l'app avec **BrowserRouter**, on déclare **<Routes>** et des **<Route>**. **Link** remplace **<a>** pour ne pas recharger la page.

useNavigate, **useParams**, **useLocation** couvrent 90 % des besoins du TP. Un layout avec **Outlet** garde le header visible.

Exercice guidé : nav Accueil / Favoris / À propos, page 404.

- Déclaration des routes
- Navigation déclarative (Link)
- Navigation impérative (navigate)
- Layout + Outlet

```jsx
import { BrowserRouter, NavLink, Outlet, Route, Routes, useNavigate } from 'react-router-dom';

function Layout() {
  return (
    <>
      <nav>
        <NavLink to="/" end>Accueil</NavLink>
        <NavLink to="/favoris">Favoris</NavLink>
      </nav>
      <Outlet />
    </>
  );
}

function Home() {
  const navigate = useNavigate();
  return (
    <button type="button" onClick={() => navigate('/pays/FR')}>
      Ouvrir la France
    </button>
  );
}

export function RouterWorkshop() {
  return (
    <BrowserRouter>
      <Routes>
        <Route element={<Layout />}>
          <Route index element={<Home />} />
          <Route path="pays/:code" element={<p>Fiche</p>} />
          <Route path="favoris" element={<p>Favoris</p>} />
          <Route path="*" element={<p>404</p>} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

Mise en application : guide de pays

TP fil rouge final : construire **WorldGuide** avec React Router et REST Countries.

Parcours : liste filtrable, fiche **/pays/:code** (drapeau, capitale, population, langues), liens vers pays frontaliers, page favoris, 404 si code inconnu.

Bonus : search params **?region=**, lazy de la carte, tests RTL du routage, Query pour le cache.

- Routes : /, /pays/:code, /favoris, *
- API : restcountries.com
- État local + localStorage favoris
- Livrable : SPA navigable et partageable

```
import { useEffect, useMemo, useState } from 'react';
import { Link, Route, Routes, useParams } from 'react-router-dom';
import { countriesApi } from './react_api_clients.js';
import { useFavorites } from './react_useState.js';

function CountryListPage() {
  const [countries, setCountries] = useState([]);
  const [query, setQuery] = useState('');
  const { codes, toggle } = useFavorites();

  useEffect(() => {
    countriesApi.all().then(setCountries);
  }, []);

  const visible = useMemo(() => {
    const q = query.toLowerCase();
    return countries.filter((c) => c.name.common.toLowerCase().includes(q));
  }, [countries, query]);

  return (
    <section>
      <h1>WorldGuide</h1>
      <input value={query} onChange={(e) => setQuery(e.target.value)} placeholder="Filtrer"
/>
      <ul>
        {visible.map((c) => (
          <li key={c.cca2}>
            <Link to={`/pays/${c.cca2}`}>{c.name.common}</Link>
            <button type="button" onClick={() => toggle(c.cca2)}>
              {codes.includes(c.cca2) ? '★' : '☆'}
            </button>
          </li>
```

```
        ))}
      </ul>
    </section>
  );
}

function CountryDetailPage() {
  const { code } = useParams();
  const [country, setCountry] = useState(null);

  useEffect(() => {
    countriesApi.byCode(code).then((data) => setCountry(Array.isArray(data) ? data[0] :
data));
  }, [code]);

  if (!country) return <p>Chargement…</p>;
  const borders = country.borders ?? [];

  return (
    <article>
      <Link to="/">← Liste</Link>
      <h1>{country.name.common}</h1>
      <p>Capitale : {country.capital?.[0]}</p>
      <p>Population : {country.population}</p>
      <h2>Frontières</h2>
      <ul>
        {borders.map((border) => (
          <li key={border}>
            <Link to={`/pays/${border}`}>{border}</Link>
          </li>
        ))}
      </ul>
    </article>
  );
}

export default function WorldGuideApp() {
  return (
    <Routes>
      <Route index element={<CountryListPage />} />
      <Route path="pays/:code" element={<CountryDetailPage />} />
    </Routes>
  );
}
```